Szakdolgozat

Miskolci Egyetem

Szkriptelési megoldások a játékfejlesztésben

Készítette:

Lengyel Gábor

Programtervező informatikus

Témavezető:

Dr. Mileff Péter

Miskolc, 2026

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

Szakdolgozat Feladat

Lengyel Gábor (GKIU70) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: Számítógépes játékfejlesztés

A szakdolgozat címe: Szkriptelési megoldások a játékfejlesztésben

A feladat részletezése:

A hallgató feladata, hogy áttekintse a játékfejlesztésben jelenleg elérhető legismertebb szkriptelési technikákat és megoldásokat. Mutasson be néhány olyan létező játékot, amely ilyen technikán alapul. Tervezzen olyan saját játékmotort, amely valamilyen modern szkriptelési (akár egyedi) megoldásra épül, lehetővé teszi a játék bizonyos funkcióinak konfigurálását, szkriptelését, vezérlését. Implementálja az alkalmazást, majd részletesen mutassa be a megvalósítás lépéseit és az elért eredményeket.

Témavezető: Dr. Mileff Péter, egyetemi docens

A feladat kiadásának ideje: 2024. szeptember 20.

. . . . . .
szakfelelős

Eredetiségi Nyilatkozat

Alulírott **Lengyel Gábor**; Neptun-kód: `GKIU70` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *Szkriptelési megoldások a játékfejlesztésben* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ... év ..... hó ..... nap

.....

Hallgató

1. A szakdolgozat feladat módosítása

szükséges (módosítás külön lapon)
nem szükséges

....
dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás):
.....
.....
.....

konzulens (dátum, aláírás):
.....
.....
.....

3. A szakdolgozat beadható:

....
dátum témavezető(k)

4. A szakdolgozat ..... szövegoldalt
..... program protokollt (listát, felhasználói leírást)
..... elektronikus adathordozót (részletezve)
.....
..... egyéb mellékletet (részletezve)
.....
tartalmaz.

....
dátum témavezető(k)

5. A szakdolgozat bírálatra

bocsátható
nem bocsátható

A bíráló neve: .....

....
dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata: .....
a bíráló javaslata: .....
a szakdolgozat végleges eredménye: .....

Miskolc, ...
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés **1**

2. A videójátékok története **2**

3. A játékfejlesztés folyamata **4**
3.1. Szoftverfejlesztési életciklus 4
3.1.1. Követelmények meghatározása 4
3.1.2. Tervezési fázis 5
3.1.3. Pre-produkciós fázis 5
3.1.4. Gyártás 6
3.1.5. Tesztelés 6
3.1.6. Karbantartás 7
3.2. A játékmotor 7
3.2.1. Unreal Engine 7
3.2.2. Unity 8
3.2.3. Godot 8
3.2.4. Egyedi játékmotorok 8

4. A szkriptelés szerepe a játékfejlesztésben **9**
4.1. Mi a szkriptelés? 9
4.2. A programozás és a szkriptelés összehasonlítása 9
4.3. A szkriptelés előnyei 10
4.4. A szkriptelés korlátai 11
4.5. Szkriptnyelvek 11
4.5.1. JavaScript 11
4.5.2. Lua 11
4.5.3. Ruby 12
4.5.4. Egyedi szkriptnyelvek 12
4.6. Szkriptelési megoldások 12
4.6.1. Végrehajtási mechanizmusok 12
4.6.2. A szkript és a játékmotor kapcsolata 13
4.7. Szkriptelhető játékelemek 14
4.7.1. Játékmenet 14
4.7.2. Felhasználói felület 14
4.8. Példák szkriptelést alkalmazó játékokra 14
4.8.1. Don't Starve 14
4.8.2. World of Warcraft 15
4.8.3. Roblox 15
4.8.4. Grand Theft Auto játékok 16

5. Fejlesztési dokumentáció **18**
5.1. Követelmények meghatározása . . . 18
5.1.1. Funkcionális követelmények . . . 18
5.1.2. Nem-funkcionális követelmények . . . 19
5.2. Tervezés . . . 19
5.2.1. Időbeosztás és erőforrás-tervezés . . . 20
5.2.2. A játékmotor fejlesztési mérföldkövei . . . 20
5.2.3. Fejlesztési módszertan . . . 20
5.2.4. A játékmotor funkcionális terve . . . 21
5.2.5. A szkriptelési keretrendszer terve . . . 22
5.2.6. Tervezett prototípusok . . . 22
5.3. A fejlesztőkörnyezet és technológiák bemutatása . . . 22
5.3.1. C programozási nyelv . . . 22
5.3.2. Lua szkriptnyelv . . . 23
5.3.3. OpenGL . . . 23
5.3.4. SDL . . . 24
5.3.5. További felhasznált könyvtárak . . . 24
5.3.6. Fordítási környezet és eszközök . . . 24
5.4. A játékmotor implementálása . . . 25
5.4.1. Az alkalmazás felépítése és működési ciklusa . . . 25
5.4.2. A színtér . . . 26
5.4.3. Kamerakezelés és irányítás . . . 26
5.4.4. Fények . . . 30
5.4.5. Objektum- és textúrakezelés . . . 31
5.4.6. Shaderek . . . 34
5.4.7. Ütközésérzékelés és fizika . . . 36
5.4.8. Domborzat . . . 39
5.4.9. Részecskerendszer . . . 41
5.4.10. Felhasználói felület (UI) . . . 43
5.4.11. Hangrendszer . . . 44
5.5. A szkriptelési rendszer implementálása . . . 44
5.5.1. A C és a Lua összekapcsolása . . . 44
5.5.2. Eseménykezelés . . . 46
5.5.3. Adatkezelés . . . 46
5.5.4. Időzítők . . . 47
5.5.5. Zónák . . . 47
5.5.6. Felvehető tárgyak . . . 48
5.5.7. Részecskerendszer . . . 48
5.5.8. Erőforrások és az erőforrás-kezelő . . . 49
5.5.9. Pályabetöltés . . . 50
5.5.10. Fejlesztői konzol . . . 51
5.6. A keretrendszer használata . . . 52
5.6.1. A menürendszer . . . 52
5.7. Tesztelés . . . 52
5.7.1. Funkcionális tesztelés . . . 52
5.7.2. Nem-funkcionális tesztelés . . . 52

6. Összefoglalás **53**

1. fejezet

Bevezetés

A játékfejlesztés a szoftverfejlesztés egyik leggyorsabban növekvő és legösszetettebb ága. Az informatikának ezen területe olyan szakterületeket ölel fel, mint a számítógépi grafika, a fizikai szimuláció vagy a mesterséges intelligencia. Emellett számos művészeti és kreatív elem is kiemelt szerepet játszik egy videójáték elkészítésében. Központi szerepet kaphat a történetmesélés, a zeneszerzés, sőt még a színészi munka is. Éppen ezért egy videójáték létrehozása legalább annyira művészet, mint a festészet, a zeneszerzés vagy a filmművészet.

Az elmúlt évtizedekben a játékszoftverek rohamos fejlődésen mentek keresztül. Míg a kezdeti programok egyszerű grafikával és korlátozott funkciókkal rendelkeztek, a mai játékok már a valósághoz közeli vizuális élményeket és gazdag, sokrétű játékmenetet kínálnak. A technológia fejlődése lehetővé tette az élethű fizikai szimulációkat és interaktív környezetek részletgazdag megalkotását. Ezzel párhuzamosan megnőtt az igény az olyan technológiák iránt, amelyek felgyorsítják a munkafolyamatokat.

A modern játékfejlesztésben a különböző szkriptelési megoldások nemcsak a fejlesztést és tesztelést teszik egyszerűbbé, hanem jelentősen növelik a testreszabhatóságot és a rugalmasságot is. A szkriptelés legnagyobb előnye, hogy a játék bizonyos részei dinamikusan módosíthatók anélkül, hogy a fejlesztőknek ismerniük kellene az alacsony szintű játékmotor működését, illetve annak újrafordítása nélkül is életbe lépnek a szkriptek által meghatározott módosítások.

Számos olyan népszerű alkotás létezik napjainkban, amelynek sikerét a szkriptelési lehetőségek tartják fenn. Például a játékosok olyan egyedi játékszerverekhez csatlakozhatnak, amelyeknek a műfaja és minősége kizárólag a közösség kreativitásán múlik. Ezeket a szkriptelési folyamatokat dokumentációk és API-k segítik, így alapszintű programozási ismeretekkel is izgalmas rendszerek építhetők. Az elmúlt években jelentős időt töltöttem el ilyen környezetekben játékosként és fejlesztőként egyaránt, ami motivált arra, hogy elmélyítsem a tudásomat a témában.

A szakdolgozat célja, hogy bemutassa a szkriptelés szerepét a videójáték-fejlesztésben. A dolgozat gyakorlati részében bemutatásra kerül egy saját fejlesztésű, C és Lua nyelvekre épülő játékmotor és szkriptelési keretrendszer. Kitűzött cél még egy olyan rugalmas, erőforrás- és eseményvezérelt szkriptrendszer tervezése és kialakítása, amely segítségével teljesen különböző, akár eltérő műfajú játékok is készíthetők.

2. fejezet

A videójátékok története

A videójátékok a szórakozás egy formáját jelentik, amely valamilyen elektronikus eszköz, például számítógép, konzol, mobiltelefon vagy egyéb hardver használatával történik. Ahogy az informatika és később az internet is egyre inkább átalakította a világot és jelentős mértékben felgyorsította, valamint új lehetőségekkel bővítette ki az életünket, úgy a videójátékok is új kikapcsolódási formákat kínálnak számtalan formában és műfajban.

A videójátékok évről évre egyre népszerűbbek, és mára egy meghatározó iparággá nőtték ki magukat, amely minden nem és korosztály számára szórakozási lehetőséget biztosít. Aki nyitott a videójátékokra, biztosan találhat olyan stílusú játékot, amiben szívesen elmerül: elérhetők számítógépes, játékkonzolos, mobiltelefonos játékok is a történelmi, stratégiai játékoktól kezdve az interaktív, filmszerű és történetközpontú játékokon át az alkalmi, ügyességi játékokig.

A videójátékok története egészen a 20. század közepéig nyúlik vissza. Ebben az időszakban alkották meg az első videójátékokat, melyek persze még meglehetősen kezdetlegesek voltak mai szemmel nézve. Ezek ekkor még inkább tudományos kísérletek és bemutatók voltak, nem pedig széles körben elérhető szórakoztató termékek [9]. Az 1958-os Tennis for Two és az 1962-es Spacewar! tekinthetők a legelső videójátékoknak [3].

Az 1970-es és 1980-as években jelentek meg az első játéktermek, amelyek hatalmas népszerűségnek örvendtek. Itt olyan klasszikus játékokkal lehetett játszani, mint a Pong, a Pac-Man, vagy a Space Invaders. Ezzel párhuzamosan kezdték el árusítani az otthoni konzolokat is. 1972-ben jelent meg a Magnavox Odyssey, az első otthoni játékkonzol, majd a 70-es évek végén az Atari 2600 is bekerült a boltokba, amely már cserélhető játékkazettákkal működött [9].

Az 1980-as évek közepén a Nintendo NES (Nintendo Entertainment System) hozta el a nagy áttörést, miután a videójáték-ipar egy rövid időre összeomlott. A NES forradalmasította a piacot, és olyan ikonikus játékokat hozott el a háztartásokba, mint a Super Mario Bros. és a The Legend of Zelda [3]. Később, a Sega a Genesis (vagy Mega Drive) konzollal és a Sonic the Hedgehog játékkal kihívója lett a Nintendónak, amely egy hatalmas rivalizálást indított el a két vállalat között [9].

Az 1990-es években történt meg a 3D-s forradalom. A Doom és a Quake elterjesztették az FPS (belső nézetű lövöldözős) műfajt [3]. A Sony által kiadott PlayStation és a Nintendo 64 lehetővé tették a 3D grafika megjelenítését konzolokon is [9]. Ekkoriban terjedt el a CD-ROM formátum, ami nagyobb tárhelyet biztosított a játékoknak a hagyományos kazettákhoz képest [3].

A 2000-es években az internet elterjedésével egyre népszerűbbek lettek az online többjátékos (multiplayer) játékok is. Olyan játékok jelentek meg, mint a máig aktív World of Warcraft (WoW), valamint elindult az Xbox Live és a PlayStation Network, amelyek lehetővé tették az online játékot és a digitális tartalomvásárlást a konzolokon [9].

Az okostelefonok megjelenése és a mobilinternet fejlődése is óriási változást hozott, az alkalmi játékosok számára is elérhetővé tette a videójátékokat, és ezzel elindult a mobiljátékok hatalmas robbanása is [3].

A technológia folyamatosan fejlődött, így egyre élethűbb grafikát, hanghatásokat és összetett játékmeneteket hoztak létre a játékokban. Ezt a technológiát korábban is használták már filmek gyártásakor, a videójátékokban azonban csak az elmúlt években honosodott meg. A jelenleg is rohamosan fejlődő mesterséges intelligencia a játékfejlesztés sok pontján alkalmazható, és ma már egyre inkább alkalmazzák is. A nem játékos által irányított karakterek viselkedése, a procedurális tartalomgenerálás, de akár a hangfelismerés és az ez alapján történő interaktív párbeszédek is jó példák arra, hogy milyen formában lehet jelen a mesterséges intelligencia egy videójátékban [13].

A virtuális valóság is egy teljesen új fejezetet nyitott a videójátékok történetében. A VR új szintre emelte a játékélményt, lehetővé teszi azt, hogy a játékosok közvetlenül részesei legyenek a játéknak. A VR, illetve a mesterséges intelligencia fejlődése is nagy szerepet játszhat a videójátékok jövőjében [3].

A videójátékok ma már nem csupán kikapcsolódási lehetőséget jelentenek, hanem hatalmas szerepük van a közösségek kialakulásában is. Számtalan videójátékokon alapuló közösség jött létre különböző platformokon, ahol a játékosok új kapcsolatokat építhetnek ki, csapatokat alkothatnak és közösen vehetnek részt különböző eseményeken. Az e-sport szerepe is jelentős mértékben megnőtt az elmúlt években: professzionális játékosok és csapatok mérkőznek meg egymással különböző bajnokságokon, amiket rajongók milliói követnek figyelemmel [9].

3. fejezet

A játékfejlesztés folyamata

3.1. Szoftverfejlesztési életciklus

Egy szoftver fejlesztése során, különösen egy olyan komplex folyamat esetén, mint a játékfejlesztés, nagyon fontos egy fejlesztési struktúrát követni, amiben egyértelműen meghatározhatók a fejlesztés fázisai, és ami szabályozza a projekt menetét az első tervektől kezdve egészen a karbantartási teendőkig. Egy szoftverfejlesztési életciklus meghatározza, hogy hogyan kapcsolódnak egymáshoz logikailag és kronológiailag a fejlesztési fázisok.

Egy projektnek megfelelő módszertan kiválasztása és helyes alkalmazása számtalan előnnyel jár: leegyszerűsíti, meggyorsítja a munkát, idő- és költséghatékonyságot biztosíthat, valamint alapvetően magasabb minőségű termék előállítását teszi lehetővé.

Egy szoftver életciklusának tipikus fázisai a követelmények meghatározása, a tervezés, az implementáció, a tesztelés és a karbantartás. Megkülönböztetünk például szekvenciális, iteratív, illetve inkrementális fejlesztési modelleket [22].

A szekvenciális módszertanok, például az egyik legrégebbi fejlesztési modell, a vízesés-modell esetén a fejlesztés fázisai egymásra épülnek, korábbi fázisokra nincs visszatérés [22]. Ha mégis visszalépésre van szükség, az jelentős időigénnyel, költséggel és erőforrás-ráfordítással járhat. Az ilyen módszertanok szigorúan szabályozott projekt esetén lehetnek hatékonyak, egyébként ritkán használt a rugalmatlansága miatt. Az iteratív és inkrementális fejlesztési modellek esetében a fejlesztési folyamat iterációkban történik, és minden iterációban végeznek tesztelést, fejlesztést és tesztelést. Várhatóan minden iteráció előállít egy működő prototípust vagy termékinkremenst. Ezek a módszertanok jelentősen rugalmasabbak és agilisabbak, mint a szekvenciális módszerek [22].

3.1.1. Követelmények meghatározása

A követelmények meghatározása a projekt által megoldandó problémát határozza meg egy dokumentum formájában. Meg kell határozni a felhasználói követelményeket, a szoftver által nyújtott funkciókat, illetve további nem-funkcionális követelményeket. A szoftverminőség-jellemzők biztosítják egy szoftver minőségét. Ezekre épül a követelmény-specifikáció. Általános minőségjellemző a funkcionalitás, a megbízhatóság, a hatékonyság, a használhatóság, a karbantarthatóság és a hordozhatóság [22].

3.1.2. Tervezési fázis

Egy projektben gyakran korlátozott idő, költségvetés és erőforrás áll rendelkezésre, ezért nagyon fontos, hogy megfelelően megtervezésre kerüljön a fejlesztési folyamat. A tervezés kihagyása vagy elhanyagolása később olyan problémákhoz vezethet, amelyek megoldása jelentős időt és pénzt felemészthet. A tervezési fázis célja, hogy lefektessük az alapokat és meghatározzunk egy jó fejlesztési struktúrát.

Ebben a fázisban születik meg a projekt alapötlete, amely lehet egy narratív, egy stratégiai vagy akár egy ügyességet és reflexeket próbára tevő játék. Meghatározásra kerülnek a játék céljai, stílusa, célközönsége, valamint az alapvető játékmechanikák. Ezek azok a mechanikák, amelyek meghatározzák, hogyan működik a játék, például hogyan mozognak a karakterek, hogyan történnek az interakciók a világban, és milyen szabályok vonatkoznak a játékmenetre [18].

Fontos lépés a projekt költségvetésének és időkeretének megtervezése is. Ekkor fel kell mérni, hogy milyen erőforrások állnak rendelkezésre (például emberi erőforrások, technikai eszközök), és mennyi idő szükséges az egyes fejlesztési szakaszokhoz. A hibásan felmért költségvetés és időkorlátok akár a projekt kudarcához is vezethetnek. A tervezési fázisban alakul ki a fejlesztői csapat is, ahol minden szerepkör egyértelműen meghatározott. Egy jól működő csapatstruktúra rendkívül fontos. Ezen felül dönteni kell a programozási nyelvről, a fejlesztő eszközökről és egyéb technikai megoldásokról. A megfelelő eszköztár kiválasztása elengedhetetlen, hiszen ez az alapja a gördülékeny fejlesztésnek [22].

Szükséges egy tesztelési terv kidolgozása is, amely meghatározza, hogy a fejlesztés során mikor és hogyan zajlanak a különböző típusú tesztelések. Ez biztosítja, hogy a játék minden fázisában ellenőrizhető legyen a minőség, és a hibákat még időben felismerjük és javíthassuk. Egy játék életciklusa azonban nem ér véget a fejlesztés és tesztelés befejezésével, hiszen ezután kezdődik el a termék értékesítése, majd támogatása és karbantartása. Ezeknek a módját, idejét és helyszínét is szükséges megtervezni, ahogy az esetleges marketing eszközöket is. Mindezek figyelembevételével nem túlzó kijelentés, hogy a tervezési fázis a fejlesztés egyik legfontosabb része, hiszen egy jól kidolgozott projekt tervvel nagyobb eséllyel lesz sikeres a projekt.

3.1.3. Pre-produkciós fázis

A tervezési fázist követően részletesen kidolgozásra kerül a játék a pre-produkciós fázisban. Ebben a szakaszban konkrét tervek és prototípusok készülnek el annak érdekében, hogy a gyártási fázisra rendelkezésre állhasson minden szükséges erőforrás és információ [18].

Az egyik legfontosabb feladat ebben a fázisban a dokumentáció elkészítése. A dokumentáció tartalmazza többek között a játék funkcióinak, mechanikáinak, vizuális megjelenésének vagy éppen a játékvilág felépítésének részletes leírását. Ez biztosítja a fejlesztés gördülékenységét. Minél nagyobb precizitással készül el a dokumentáció, annál egyszerűbb és gyorsabb elvégezni magát a fejlesztést. Véglegesítésre kerülnek a választott fejlesztő eszközök, technikák és programozási nyelvek. Fontos döntést hozni arról is, hogy milyen platformra fejlesztik a játékot és milyen minőségi elvárások vonatkoznak rá. Finomításra kerülnek a tesztelési tervek is [22].

A pre-produkció során meghatározzák a fejlesztési ütemtervet, határidőket és elkészítik a költségbecslést. Ezek reális meghatározása és betartása hozzájárul a projekt sikerességéhez.

Gyakran elkészítik ebben a fázisban a játék prototípusát, ami egy korai, leegyszerűsített verziója a játéknak, amely lehetőséget ad arra, hogy a fontosabb funkciókat kipróbálják, leteszteljék. Ez sokat segít abban, hogy még a gyártás előtt felismerjék az esetlegesen felmerülő problémákat és hiányosságokat, ami sok időt és pénzt spórolhat meg. A pre-produkciós fázis sikeres lezárásával megkezdődhet a gyártási fázis, ahol a tényleges fejlesztés történik [18].

3.1.4. Gyártás

A gyártási fázisban készül el a tényleges játék, az eddig kidolgozott tervek és elgondolások implementálásra kerülnek. Ez a fejlesztési folyamat legaktívabb és legösszetettebb része. A különböző feladatokért felelős személyek vagy csapatok általában párhuzamosan tudnak dolgozni a feladataikon.

A programozók megírják a játék kódját, elkészülnek olyan rendszerek, mint például a fizikai motor, a felhasználói felület vagy a mesterséges intelligencia. Megvalósulnak a játékmechanikák, a játékvilág és az interakciók, például a karakter irányítása. A grafikusok elkészítik a játék vizuális elemeit, beleértve a környezetet, a karaktereket, az animációkat és a vizuális effekteket. Ebben a szakaszban nyerik el végleges formájukat a modellek, a textúrák és az animációk, amik a játék vizuális világát határozzák meg. A játék zenéit, hanghatásait is ki kell dolgozni. Ez magában foglalja a zenei aláfestéseket, a környezeti hangokat, karakterek hangjait és más interaktív hangelemeket.

A gyártási fázis előrehaladtával a játék egyre nagyobb részei válnak játszhatóvá, elkészülnek olyan korai verziók, mint az alfa és a béta verziók. Ezek a korai verziók lehetőséget adnak arra, hogy a fejlesztők visszajelzéseket kapjanak, és ennek megfelelően kisebb-nagyobb finomhangolást végezzenek a játékon. Amikor a játék teljesen összeállt, és már csak az utolsó simítások vannak hátra, a projekt lassan a tesztelési fázisba lép.

3.1.5. Tesztelés

A tesztelési fázisban, ahogy a neve is mutatja, a tesztelés lép előtérbe. Célja, hogy a szoftver hibamentes és stabil legyen, mielőtt publikálásra kerül. Ezt egy átfogó tesztelés biztosítja, amely alaposan vizsgálja a játék minden aspektusát.

A funkcionális tesztelés során meg kell győződni arról, hogy a játék a terveknek és a specifikációnak megfelelően működik-e. Kifejezetten azt vizsgálja, hogy a játék mechanikái, funkció helyesen valósulnak-e meg, függetlenül attól, hogy ez milyen módon történik. E tesztelés során derül fény az esetleges hibákra vagy hiányosságokra, amik miatt a játék nem elégíti ki a meghatározott követelményeket.

A nem funkcionális tesztelés vizsgálja azt, hogy mennyire jól és hatékonyan működik a szoftver. Ekkor a működés módja, nem pedig a helyessége a szempont, ellentétben a funkcionális teszteléssel. Nem funkcionális tesztelések közé tartozik a teljesítménytesztelés, a stressztesztelés, de a használhatósági és a biztonsági tesztelés is [22]. Mindezek biztosítják, hogy a játék megfelelő teljesítménnyel és stabilitással futhasson, kielégítő felhasználói élményt nyújtva.

A tesztelésnek kritikus szerepe van, hiszen a tesztelés alaposságán múlik, hogy milyen minőségű lesz az elkészültnek minősített szoftver. A játék sikerességét nagy mértékben befolyásolja a felhasználói élmény, amely nagyban összefügg a hibamentes működéssel. Egy játék esetében kifejezetten fontos, hogy ne merüljenek fel súlyos, a játékmenetet erősen zavaró vagy ellehetetlenítő hibák, mert az jelentősen befolyásol-

hatja a játék, és így a játékot készítő cég hírnevét, ami alacsonyabb bevételhez vezet rövid és hosszútávon is. Gyakori probléma, hogy egy játékot rosszul optimalizálnak, és indokolatlanul magas a rendszerkövetelménye, melyet nagyon sok játékos hardvere nem ér el. A játékos szempontjából ez a játék teljes használhatatlanságát jelenti.

Természetesen hibátlan szoftver nem létezik. Már egy közepesen komplex játék is több ezer különböző funkcióval rendelkezhet, amelyeket számtalan módon lehet kombinálni. Továbbá, a játékosok sokféle hardver- és szoftverkörnyezetben futtathatják a programot, a különböző operációs rendszerektől kezdve a változatos grafikus kártyákig és perifériákig. Ez a sok milliónyi lehetséges konfiguráció gyakorlatilag lehetetlenné teszi azt, hogy minden lehetséges helyzetben előre teszteljék a játékot.

3.1.6. Karbantartás

Abból kifolyólag, hogy hibátlan szoftver nem létezik, a legalaposabb tesztelések után is felmerülhetnek hibák. A játékok életciklusa a kiadás után sem ér véget, sőt, talán ekkor jelentkeznek a legnagyobb kihívások. A már publikált termékhez kell támogatást nyújtani, a felmerülő hibákat javítani, a teljesítményt optimalizálni, valamint a játékosok általi visszajelzéseket figyelembe véve módosításokat végrehajtani [18].

Azonban nem csak a játékélmény biztosítása és növelése az egyetlen szempont, ennél összetettebb a feladat. Egy cég szempontjából a karbantartási tevékenységek akár idő- és pénzpazarlásnak is tűnhet, hiszen a fejlesztőket fizetni kell azért, hogy javítsák a játékot, ami ráadásul jelentős időt is felemészthet. Ilyenkor a cégnek mérlegelnie kell, hogy mennyire éri meg a hibák javítása és esetleg új tartalmak bevezetése a meglévő játékokhoz, hiszen az erőforrásait akár más projektekre is fordíthatná. Nyilvánvalóan a cég számára nem csak plusz ráfordított pénzt és időt jelent ez a fázis, hanem hosszabb távú lehetőségeket is kínál. A játékosok elégedettsége és elköteleződése hosszútávon jövedelmezőbb lehet, mint azonnal egy új projektbe kezdeni, miközben elégedetlen játékosokat hagynak maguk mögött a karbantartás költségeinek megspórolása érdekében.

3.2. A játékmotor

A videójátékok létrehozásához a játékmotorok adnak keretet. Ez egy olyan szoftverkeretrendszer, amely eszközöket biztosít a játék fejlesztéséhez, és ezáltal megkönnyíti és felgyorsítja azt. Olyan alapvető funkciókat biztosít, mint például a grafikai megjelenítés, a játékfizika, hangkezelés, mesterséges intelligencia vagy felhasználói bemenetek kezelése [7].

3.2.1. Unreal Engine

Az Unreal Engine az Epic Games által fejlesztett 3D-s játékmotor. Eredetileg belső nézetű lövöldözős játékokhoz fejlesztették ki az 1990-es években, viszont azóta számos más műfajban is alkalmazták. A játékmotor C++ nyelven íródott [7]. Kimagasló grafikai képességekkel és nagyfokú testreszabhatósággal rendelkezik. Aktuális verziója, az Unreal Engine 5 2022-ben jelent meg, melynek a forráskódja elérhető GitHub-on. Ingyenesen hozzáférhető, azonban ha 1 millió amerikai dollár feletti bevételre teszünk szert a felhasználásával, abban az esetben 5%-os jogdíj illeti az Epic Gamest. Ez a díj elengedésre kerül, amennyiben a játék kizárólag az Epic Games Store-ban jelenik meg. Unreal Engine-ben készült többek között a Fortnite című játék.

3.2.2. Unity

Egy másik ismert és széles körben használt játékmotor a Unity Technologies által fejlesztett Unity. Első verziója 2005-ben jelent meg. C# programozási nyelvet használ és rendkívül sokoldalú: két- és háromdimenziós játékok készítésére is alkalmas, és számos platformot támogat, mint például PC, konzolok és mobilplatformok [7]. A mobiljátékok jelentős része a Unity játékmotorral készült. A felhasználói felülete egyszerű és intuitív, így kezdők számára különösen jó választás lehet. A Unity egy sokoldalú és könnyen hozzáférhető motor, azonban teljesítmény szempontjából hátrányban lehet az Unreal Engine-nel szemben [7]. Unity motort használ a megjelenésekor igencsak felkapott Pokémon GO telefonos játék is.

3.2.3. Godot

A Godot egy nyílt forráskódú, ingyenes és platformfüggetlen játékmotor, ami 2D-s és 3D-s fejlesztésre is alkalmas. Teljesen ingyenes és licencdíjaktól mentés, és mivel nyílt forráskódú, szabadon módosítható. A Godot-ban fejlesztett játékok C#-ban, vagy a motor saját nyelvében, GDScriptben vannak írva. A GDScript egy magas szintű, Pythonhoz hasonló nyelv, amely lehetőséget ad a jobb optimalizációra. A motor nem csak kódolásra, hanem grafikus felületen történő kódszerkesztésre is ad lehetőséget Visual Scripting néven. Folyamatosan fejlődik, bár 3D-s képességei még nem érik el az Unreal Engine vagy a Unity szintjét. Kifejezetten népszerű az indie fejlesztők körében [7].

3.2.4. Egyedi játékmotorok

Egy játék fejlesztése során dönthetünk úgy, hogy nem egy meglévő játékmotort használunk, hanem egy teljesen saját keretrendszert írunk. Ez a megközelítés számos előnyt kínál. Az egyedi motor teljes mértékben a fejlesztők kontrollja alatt áll és a projekthez alkalmazkodik. Nem tartalmaz felesleges funkciókat, kizárólag a projekt igényeit elégíti ki, ezáltal jobb optimalizációt nyújt. Könnyebben megvalósíthatók egyedi ötletek, mechanikák vagy megjelenési stílusok, hiszen nem kell aggódni a motor korlátai miatt. Továbbá az egyedi motor mentes a licencdíjaktól, ez pedig hosszú távon költségmegtakarítást jelenthet [7].

A számos előny mellett ugyanakkor a hátrányokat is figyelembe kell venni. Egy játékmotor megírása rendkívül összetett, idő- és erőforrásigényes feladat, és nagyobb szaktudást igényel. Egy túlságosan specifikus játékmotor esetén fennáll annak a veszélye, hogy egy új projekthez nem lesz alkalmas, így jelentős módosításokra vagy akár teljesen új motor fejlesztésére lehet szükség. A jól ismert, bevált játékmotorok egyik nagy előnye a közösségi támogatás és a széleskörű dokumentációk elérhetősége, ami egy saját motor esetén nincs jelen. A tesztelés, a hibakeresés és a hibák javítása is kizárólag a fejlesztőcsapatot terheli egy egyedi motor esetében [16].

A népszerű Rockstar Games a saját RAGE motorját használja játékaihoz, mint például a Grand Theft Auto vagy a Red Dead Redemption címekhez. A CD Projekt RED, a jól ismert Witcher és Cyberpunk 2077 játékok fejlesztői pedig a REDengine nevű játékmotorjukat használja.

Összegezve az egyedi játékmotorok létrehozása komoly kihívásokkal jár, azonban ha meg tudunk birkózni ezekkel és ki tudjuk aknázni a potenciális előnyöket, akkor azok messze meghaladják a hátrányait.

4. fejezet

A szkriptelés szerepe a játékfejlesztésben

A szkriptelés lehetővé teszi, hogy a fejlesztők gyorsabban és rugalmasabban valósítsák meg a játékok egyes funkcióit. Leegyszerűsíti a komplexebb rendszereket és magasabb szintű vezérlést tesz lehetővé anélkül, hogy magában a játékmotorban kéne módosításokat végrehajtani [7].

4.1. Mi a szkriptelés?

A szkriptelés során olyan utasítássorozat készül, amelyet általában valamilyen szkriptnyelven írnak azzal a céllal, hogy vezérelje egy program működését vagy automatizálja a program bizonyos funkcióit.

A szkriptek magas szintű, erősen absztrakt kódok, amelyek megkönnyítik az összetettebb rendszerek vezérlését anélkül, hogy mélyebb, alacsony szintű programozást igényelnének [16]. Az absztrakció gyakran olyan mértékű is lehet, hogy nincs is szükség kódírásra: ha például a rendszer biztosít egy grafikus felhasználói felületet, amelyen keresztül egyszerűen gombnyomásokkal beállíthatók a kívánt funkciók, az is szkriptelésnek tekinthető. Szkriptelés lehet az operációs rendszer parancssorába beírt utasítások sorozata is.

Nem véletlen, hogy éppen „szkriptnek” nevezik, ami eredetileg forgatókönyvet jelent. A szkriptelés során nem bővül a program eszköztára, hanem a már meglévő eszközöket és lehetőségeket használja fel a kívánt funkciók megvalósításához. A szkript egy „forgatókönyvet” ad a programhoz: meghatározhatják, hogy mikor, mi és hogyan történjen [7]. A szkriptek tehát lehetővé teszik, hogy a fejlesztők gyorsan és egyszerűen alakítsák egy program egyes funkcióit, miközben az alapvető működéséhez nem nyúlnak hozzá.

4.2. A programozás és a szkriptelés összehasonlítása

Bár ránézésre nem feltétlenül látni rögtön a különbséget, jelentősen eltér egymástól a programozás és a szkriptelés. Mindkettő a szoftverfejlesztés eszköze, azonban különböző céljaik vannak.

A programozás célja a szoftverek alapvető funkcióinak létrehozása, amely mélyebb ismereteket és alaposabb tervezést követel meg. Programozás során történik a mű-

ködési logika, az adat- és fájlkezelés, valamint hardverközeli funkciók megvalósítása. Programozási nyelvek közé tartozik például a C, a C++ vagy a Java. Ezek a nyelvek általában alacsonyabb szintű hozzáférést biztosítanak a számítógép hardvereihez és erőforrásaihoz, így nagyobb szabadságot kapnak a fejlesztők [1].

Ezzel szemben a szkriptelés egy magasabb szintű réteget biztosít, amely nem módosítja a program alapvető működését, hanem annak bizonyos meglévő részeit vezérli. Célja a gyors és rugalmas fejlesztés. Gyakran használt szkriptnyelv a JavaScript vagy a Lua.

A programozási nyelvek szintaktikája általában bonyolultabb, mint a szkriptnyelveké, mivel a szkriptnyelvek magasabb szintű absztrakciót nyújtanak. Ez azt jelenti, hogy a szkriptnyelvek egyszerűsített, könnyebben érthető parancsokat használnak, amelyek sokkal közelebb állnak a természetes nyelvhez [7]. Ezáltal szkriptelni jelentősen könnyebb és intuitívabb, mint egy programot megírni, amelynek a nyelve sokszor bonyolult szintaktikával, szabályokkal és konvenciókkal rendelkezhet.

A programozási nyelvek jellemzően fordított (compiled) nyelvek, míg a szkriptnyelvek inkább értelmezettek (interpreted). Ez a különbség jelentősen befolyásolja többek között a futási időt. A fordított nyelvek esetén a megírt kódot egy fordító (compiler) gépi kódra fordítja futás előtt, ami így a processzoron már közvetlenül futtatható. A fordítási folyamat ugyan időigényes lehet, viszont ezt követően a program már gyorsabban fut, mivel nem igényel további értelmezést [1].

Ezzel szemben az értelmezett nyelvek egy értelmezőn (interpreteren) keresztül futnak. A fő különbség a fordítással szemben az, hogy míg a fordítás megtörténik a futtatás előtt, addig a szkriptek értelmezése és végrehajtása a futás közben, alapesetben soronként történik. Ez általában lassabb futási időt eredményez, viszont nincs szükség előzetes fordításra [1].

4.3. A szkriptelés előnyei

A szkriptnyelvek egyszerűsége miatt a szkriptelés könnyen tanulható és alkalmazható, ami gyorsabb fejlesztést tesz lehetővé. Egy kisebb szkript megírása, módosítása és tesztelése jelentősen egyszerűbb, mint egy nagyobb programé, amely fordítást igényel. Egyszerűsége és rugalmassága miatt a szkriptelés kifejezetten ideális prototípusok gyors létrehozására [16].

A szkriptnyelvek egyszerű és jól olvasható szintaxisa lehetővé teszi, hogy a kezdők is gyorsan el tudják sajátítani a használatukat. Gyorsabb beilleszkedést biztosít a fejlesztői környezetbe, hozzájárul a hibák számának csökkentéséhez és megkönnyíti a kód karbantartását, de hatékonyabb munkamegosztást is lehetővé tesz.

A szkriptelés rendkívül rugalmas, testreszabható és újrahasznosítható megoldásokat tesz lehetővé. A szkriptnyelvek gyakran platformfüggetlenek, vagyis ugyanaz a szkript több különböző környezetben is futtatható minimális módosítás után [1]. Mivel a szkriptek általában interpretált működésűek, gyorsan frissíthetők és azonnal tesztelhetők.

Jól megírt szkriptek segítségével egyszerűen létrehozhatók moduláris kódrészek, amelyet igény szerint fel lehet használni többször is, akár más projektekben is [7]. A szkriptek alkalmasak arra, hogy összekapcsoljanak több különféle rendszert vagy alkalmazást, például végezhet műveleteket egy weboldal és egy adatbázis között.

Összességében a szkriptelés hatékony és rugalmas megoldás, amely lehetővé teszi, hogy a fejlesztők gyorsan tudjanak alkalmazkodni a változó követelményekhez és egyszerűbben tudják kezelni az összetettebb fejlesztési feladatokat.

4.4. A szkriptelés korlátai

A szkriptelés általában magas szinten történik, így nem biztosít lehetőséget memóriakezelésre és más, hardver-közeli funkciók elvégzésére [7]. Mivel a szkript egy másik programtól függ, ami értelmezi azt, így abban az esetben, ha a program hibásan működik, a szkript sem az elvárásoknak megfelelően fog működni. Ez még nagyobb problémát jelent, ha a program forráskódját nem tudjuk saját kezűleg javítani, akár azért, mert nincs hozzáférésünk vagy nem vagyunk elég képzettek hozzá.

Az interpretáció miatt a kód végrehajtása futás közben történik, ami hosszabb futási időket eredményezhet, szemben a fordított nyelvekkel. Ez leginkább nagy, összetett és erőforrás-igényes feladatok esetén lehet jelentős [1]. Szintén az interpretált módon történő futás miatt a hibák csak futásidőben jelentkeznek, ami megnehezíti a hibakeresést, illetve a hibák elhárítását.

Az interpretált működés biztonsági kockázatokat is hordozhat. Különösen ügyelni kell a megfelelő védelem kialakítására, hogy elkerülhetők legyenek bizonyos támadások, például az injekciós támadások. Ilyen például az SQL-injekció. Ezeknek a támadásoknak a lényege az, hogy valamilyen sebezhetőséget (például rosszul kezelt inputokat) kihasználva saját kódokat „injektálnak” a kódba futás közben, gyakran ártó szándékkal [22].

Összegezve a szkriptelésnek sok előnye van, mint például az egyszerűség, a rugalmasság és a modularitás, azonban a programozással szemben hátrányban lehet teljesítmény, erőforrásokhoz való hozzáférés, biztonság és hibakeresés szempontjából is.

4.5. Szkriptnyelvek

4.5.1. JavaScript

A JavaScript a legismertebb és legelterjedtebb szkriptnyelvek egyike, melyet azzal a céllal hoztak létre, hogy a webfejlesztésben kiegészítse a HTML és a CSS funkcionalitását. Lehetővé teszi dinamikus, interaktív tartalmak, például űrlapok vagy animációk létrehozását. A JavaScript egy egyszerű, platformfüggetlen nyelv, szintaxisa hasonló a Java programozási nyelvhez. Rengeteg dokumentáció és oktatóanyag áll rendelkezésre, amely jelentősen megkönnyíti a tanulási és fejlesztési folyamatokat. A nyelv webalapú játékok írására is alkalmas, például olyan keretrendszerek használatával, mint a Phaser vagy a Three.js. Emellett JavaScript alapú mobil- és asztali alkalmazások is léteznek.

4.5.2. Lua

A Lua egy könnyű és gyors, C nyelven íródott szkriptnyelv, aminek célja az volt, hogy beágyazható legyen más programokba [8]. Támogatja az egyszerű és rugalmas kódolást, más szkriptnyelvekhez képest pedig kiemelkedő a sebessége, a bővíthetősége és a kis mérete. Különösen vonzó játékfejlesztési szempontokból, hiszen úgy tervezték, hogy egyszerűen integrálható legyen más alkalmazásokba. Segítségével gyorsan és nagyobb erőfeszítések nélkül készíthetünk új funkciókat vagy módosíthatunk meglévőket, az alapmotor kódja módosítása nélkül [7]. A Lua közösségi támogatása alacsonyabb a JavaScripthez képest és korlátozottabbak a könyvtárai, így sok funkciót a fejlesztőknek kell elkészíteniük, vagy külső könyvtárakat kell használnia.

4.5.3. Ruby

A Ruby egy teljesen objektumorientált, dinamikus, általalános célú nyelv. Különösen népszerű a webfejlesztésben, de a játékfejlesztés területén is alkalmazható. A kódja egyszerű és olvasható, ugyanakkor alkalmas komplex funkciók hatékony megvalósítására is. A dinamikus adattípusok és az automatikus memóriakezelés rugalmassá teszi. Interpreterének köszönhetően platformfüggetlen [1]. A Ruby nyelvnek jelentősebb teljesítménybeli korlátai vannak, mint például a Lua-nak, ami miatt bizonyos projekteknél nem feltétlenül ideális választás. Játékfejlesztés szempontjából kevésbé elterjedt, így korlátozott támogatás és dokumentáció érhető el ezen a területen.

4.5.4. Egyedi szkriptnyelvek

Egy sajátos igényekkel rendelkező projekt esetén szükséges lehet egy egyedi szkriptnyelv fejlesztése, hogy biztosítva legyenek a megfelelő megoldások a felmerülő igényekre. Egy egyedi szkriptnyelv testre szabható, a projekt szükségletei szerint alakítható ki. Emiatt egyszerűbb és jobb teljesítményt nyújthat, miközben kielégíti a különleges funkciók implementációjának igényeit. A nyelv megtervezése és implementálása azonban jelentős idő- és erőforrás ráfordítást igényel, kompatibilitási problémák merülhetnek fel, valamint tanulási nehézségeket is okozhat. Számos játékmotor kínál saját szkriptnyelvet. Például a Godot által használt GD-Script lehetővé teszi a motor egyszerű és hatékony kezelését [7].

4.6. Szkriptelési megoldások

4.6.1. Végrehajtási mechanizmusok

Míg a játékmotorok gyakran C- vagy C-szerű nyelven íródnak, melyek a fordítás után közvetlenül a hardveren futnak natív gépi kódként, addig a szkriptek jellemzően magasabb szintű végrehajtási rétegen keresztül kerülnek futtatásra, például interpreter vagy virtuális gép segítségével. A szkriptek futtatásának klasszikus, legegyszerűbb módja az interpretált végrehajtás, amely során a kód futás közben kerül feldolgozásra, például egy játékmotorba integrált interpreter által. Ez nagy rugalmasságot ad, hiszen a szkriptek módosítása nem igényel újrafordítást és futás közben is módosíthatók. A játékfejlesztésben ez kifejezetten előnyös a prototípus-készítés során, valamint megkönnyíti a gyors finomhangolásokat és a tesztelést. Ez azonban több számítást igényel, ami lassabb végrehajtáshoz vezet [1].

A modern játékfejlesztésben az interpretált végrehajtás által nyújtott teljesítmény gyakran nem kielégítő. Ezért terjedtek el a virtuális gép alapú megoldások. Ilyenkor a forráskód nem közvetlenül futás közben kerül feldolgozásra, hanem előzetesen egy úgynevezett bájtkódra fordul [1]. A bájtkód egy platformfüggetlen, köztes utasításkészlet, amelyet a virtuális gép értelmez és hajt végre. Mivel a virtuális gépnek már nem a bonyolult, ember által írt forráskódot kell értelmeznie, hanem egy előre feldolgozott egyszerűbb kódot, így általában jelentős teljesítményjavulás érhető el, de természetesen még mindig lassabb, mint a natív fordítás. Emellett megőrzi az interpretált végrehajtás rugalmasságát is [7].

A teljesítmény további növelésére alkalmas módszer a futásidejű fordítás (Just-In-Time compilation, JIT). Számos modern játékmotor alkalmazza ezt a modellt. Lényege,

hogy a futtatókörnyezet a szkripteket (vagy azok gyakran használt, teljesítménykritikus részeit) futás közben natív kódra fordítja. Ez egy hibrid megoldás, amely ötvözi a gyors iteráció lehetőségét a natívhoz közeli futási sebességgel. Ez a módszer azonban a program indulásakor több időt vesz igénybe, továbbá megnövekedett processzor és memóriahasználat jellemzi. Biztonsági szempontból is felmerülnek megfontolandó kérdések. A JIT-nek írható és végrehajtható memóriaterületre van szüksége a kód tárolásához, ami potenciális sebezhetőséget jelenthet. Ez megsérti az úgynevezett Write XOR Execute elvet, melynek lényege, hogy egy adott memórialap vagy csak írható, vagy csak végrehajtható lehet, de sosem mindkettő egyszerre. Ennek betartása alapvető védelmet nyújt a kódinjektáláson alapuló támadásokkal szemben, ezért számos zárt platform operációs rendszere korlátozza vagy tiltja a JIT használatát [1].

Ahead-Of-Time (AOT) fordítás esetén már a build folyamat során megtörténik a natív kódra való fordítás, így futáskor nem történik értelmezés, bájtkód-feldolgozás és fordítás sem. Ezzel a megközelítéssel a futási teljesítmény közel megegyezik a natív kód teljesítményével, miközben kiküszöbölhetők a JIT biztonsági kockázatai és a futás közbeni megnövekedett erőforrás-igény. Fontos azonban megjegyezni, hogy az AOT használatával elveszítjük a szkriptnyelvek egyik legnagyobb előnyét, a fejlesztési rugalmasságot. Mivel minden szkriptmódosítás érvényesítéséhez újrabuildelés szükséges, a fejlesztési ciklus lelassul. Ennek ellenére az AOT fordítás széles körben elterjedt olyan platformokon, ahol a teljesítmény és a biztonság elsődleges szempont [7].

A legtöbb mai játékmotor hibrid megközelítést követ. A fejlesztés során ezek a motorok a gyors iteráció érdekében virtuális gépet vagy JIT fordítást alkalmaznak, míg a végleges buildhez AOT fordítást használnak a maximális teljesítmény és biztonság elérése érdekében [7].

4.6.2. A szkript és a játékmotor kapcsolata

A játékmotor és a szkriptek logikailag teljesen elkülönülnek, általában eltérő nyelven írt, mégis szorosan integrált rétegek. A szkriptek célja, hogy kiterjesszék a játékmotor funkcionalitását úgy, hogy a szkriptet készítő fejlesztőnek szüksége vagy akár lehetősége se legyen a játékmotor forráskódjához való közvetlen hozzáférésre. A motor fejlesztése során meghozott döntések határozzák meg, hogy a szkriptek milyen mértékben férjenek hozzá a motor adataihoz, valamint belső, alacsony szintű rendszereihez.

A kapcsolat megteremtéséhez elengedhetetlen egy kétirányú kommunikáció kialakítása, amelyet egy API biztosít.

Egyrészt szükséges, hogy a szkript képes legyen meghívni a játékmotorban definiált függvényeket. Ehhez használhatók úgynevezett burkoló (wrapper) függvények. Ezek a függvények felelnek azért, hogy áthidalják a két eltérő programozási környezet közötti különbségeket. A motor és a szkriptnyelv eltérhet a típuskezelésükben, adatszerkezeteikben, ezért szükséges egy köztes réteg, ami az egyik környezet jellegzetességeit átkonvertálja a másik környezet által értelmezhető formátumra [8].

Másrészt azt is biztosítani kell, hogy a motor is tudjon kommunikálni a szkript felé. Ebben az esetben nem egyszerű függvényhívások történnek, hanem a motor jelzi a szkript felé bizonyos események bekövetkezését, például gombnyomásokat vagy objektumok ütközését. Ez lehetővé teszi, hogy a szkript reagálni tudjon a tőle független, a motortól érkező eseményekre. Ezt eseményvezérelt programozásnak nevezik.

4.7. Szkriptelhető játékelemek

4.7.1. Játékmenet

A játékmenet szkriptelése számos területet magában foglalhat: a játéktér kialakítását, az objektumok létrehozását és mozgatását, a fények és a hangok kezelését, a napszakok váltakozását, az ellenségek viselkedésének vezérlését, valamint az interakciókat.

Tulajdonképpen a szkriptek hozzák létre magát a játékot, a játékmotor csak biztosítja az ehhez szükséges eszközöket. Így könnyen létrehozhatók dinamikus világok, és akár számtalan különböző, egymástól teljesen független játék is felépíthető ugyanarra az alapra.

4.7.2. Felhasználói felület

A felhasználói felület szkriptelése lehetővé teszi, hogy a játékmotor újrafordítása nélkül, gyors tempóban alakítani lehessen a képernyőn megjelenő információkat. Ide tartoznak a menük vagy a játékos információt megjelenítő elemek (például életerő, pontszám).

Ebben az esetben is teljes mértékben indokolt az eseményvezérelt módon történő működés. A felhasználói felület elemei leggyakrabban a felhasználó bemenetei (például billentyűleütés, egérkattintás) hatására változtatják meg állapotukat.

4.8. Példák szkriptelést alkalmazó játékokra

A következőkben bemutatásra kerül néhány népszerű videójáték, amelyek igazolják, hogy mekkora szerepe van a szkriptelésnek a játékiparban, és milyen sokrétű felhasználási lehetőségei vannak a kis független stúdióktól kezdve a több millió dolláros AAA játékokig. Számos esetben a fejlesztők a teljes játékosbázis számára is lehetőséget biztosítottak egyedi tartalmak létrehozására az alapjátékra támaszkodva, ezzel akár évtizedekig fenntartva népszerűségüket.

4.8.1. Don't Starve

A *Don't Starve* című játék, melyet a Klei Entertainment fejlesztett, tökéletes példája a szkriptelés, valamint a játékmotor és a játéklogika szétválasztása fontosságának. Ez egy túlélőjáték, amelynek *Don't Starve Together* nevű többjátékos változata is létezik. A játék motorja egy egyedi, C++ nyelven íródott motor, amely kizárólag az alacsony szintű feladatokért felel, például az alapvető grafikai megjelenítésért, a memóriakezelésért, a fizika szimulációjáért vagy a hangkezelésért. Maga a játék, amely magába foglalja a karakter irányítását, a környezet és az ellenségek viselkedését, a világgal való interakciót és a barkácsolási lehetőségeket, Lua szkriptekkel került megvalósításra [6].

Ezzel a gyakorlatban is megvalósul a fentiekben megfogalmazott elméleti koncepció, miszerint a játékmotor önmagában csak a fejlesztéshez szükséges eszközöket biztosítja, tulajdonképpen keretrendszerként szolgál a tényleges játékot létrehozó szkriptek számára. A fejlesztők számára könnyű hordozhatóságot és gyors iterációs lehetőséget adott a Lua alkalmazása [6].

Ez a megközelítés azt is lehetővé tette, hogy a felhasználók a jól dokumentált API segítségével saját modifikációkat készítsenek. A játék úgynevezett prefabokat használ, amelyek Lua fájlokban írják le az egyes játékelemek tulajdonságait és viselkedését, és

"tervrajzként" szolgálnak arra, hogy hogyan kell létrehozni az entitásokat. Tárgyak, épületek vagy lények egyaránt ilyen prefabokként vannak megvalósítva, és a modkészítők újakat hozhatnak létre anélkül, hogy a motor kódjához hozzá kellene nyúlniuk.

Ezáltal a szkriptek használata nem csupán a fejlesztéshez és teszteléshez szükséges időt rövidítette le, hanem azt is lehetővé tette, hogy a játék hosszú éveken át aktív játékosbázissal működhessen, mivel jelentős közösség épült a felhasználók által készített tartalmakra.

4.8.2. World of Warcraft

A Blizzard Entertainment által kiadott *World of Warcraft* (WoW) az egyik, ha nem a legnagyobb hatású és egyben leghosszabb életű online szerepjáték (MMORPG). A *Don't Starve*-hoz hasonlóan C++ nyelven készült a motorja, a szkriptnyelve pedig a Lua.

A WoW addon rendszere eseményvezérelt modellen alapul. A motor eseményeket jelez (például a játékos sebződését) a szkriptek felé, amelyekre az addonok feliratkozhatnak és reagálhatnak [24]. Ez pontosan az a kommunikációs modell, amely korábban elméleti szinten bemutatásra került: a zárt motor eseményeket közvetít, a nyílt szkript pedig reagál rájuk.

A részletesen dokumentált API teszi lehetővé a játékosok számára saját kiegészítők (addonok) készítését [24]. Mivel a WoW egy kompetitív játék, a tartalomkészítő felhasználók lehetőségei korlátozottak: leginkább a felhasználói felület módosítása lehetséges. Ez nem általános szkriptkorlát, hanem a fejlesztők tudatos döntése arról, hogy a szkriptek milyen módon módosíthatják a játékot. Az API-t úgy alakították ki, hogy képes legyen olvasni a játék adatait és módosítani a felhasználói felületét, de szándékosan nem ad lehetőséget a játékmenet alapvető szabályainak módosítására (például sebzésértékek megváltoztatása) vagy a játékos döntéseinek automatizálására (például mozgás, támadás). Ennek célja az volt, hogy az addonok ne adjanak tisztességtelen előnyt.

Még ilyen korlátozott formában is megosztó az addonok használata a WoW közösségében. Sokan úgy vélik, hogy bizonyos kiegészítők már nem csupán átláthatóbb, személyre szabott felületet biztosítanak, hanem olyan előnyöket is nyújtanak (például plusz információk megjelenítésével), amelyek igazságtalannak tekinthetők más játékosokkal szemben. Sokak szerint azonban épp ez a játék legnagyobb ütőkártyája, amivel kitűnik több hasonló műfajú játék közül. Ez a vita a mai napig zajlik a közösségben, és a Blizzard több alkalommal is korlátozta vagy finomhangolta bizonyos addon-funkciók működési körét, keresve az egyensúlyt a testreszabhatóság és az igazságos játékmenet védelme között.

A World of Warcraft esete jól szemlélteti, hogy a szkriptelés szerepe nem feltétlenül a hatalmas szabadság nyújtása, ennél sokkal több szempontot kell figyelembe venni. A Blizzard tudatosan húzta meg a határt a testreszabhatóság és a játékmenet védelme között, megmutatva, hogy egy jól megtervezett szkriptelési rendszer nemcsak lehetőségeket, hanem szükséges korlátokat is magában foglal.

4.8.3. Roblox

A Roblox eltér az előzőekben bemutatott játékoktól. Ez tulajdonképpen nem is egy játék, sokkal inkább egy játékmotor és egy platform, amely teljes mértékben a felhasz-

nálók által készített tartalmakra támaszkodik [17]. A motor C++ nyelven került implementálásra, a játékmenet és interakciók logikájának kialakítására azonban a Roblox saját, nyílt forráskódú szkriptnyelvét, a Luau-t használja, amely a Lua 5.1-es verzióján alapul [11].

A Luau jelentős fejlesztéseket tartalmaz a klasszikus Lua-hoz képest. A teljesítmény javítása mellett (új bájtkód-értelmező és hatékonyabb szemétgyűjtő) a nyelv támogatja a fokozatos típuskezelést. Ez azt jelenti, hogy a fejlesztő szabadon dönthet: írhat teljesen dinamikus kódot (ahogy a hagyományos Lua-ban szokás, tehát nem adja meg a változók típusát), de ha akarja, opcionálisan típusinformációt is hozzárendelhet. Így a Roblox Studio már a kód írásakor képes észlelni bizonyos hibákat, és segítséget nyújtani, anélkül hogy a programot futtatni kellene. Mindezek mellett a nyelv teljes mértékben visszafelé kompatibilis a Lua 5.1-gyel.

A Roblox platform kliens- és szerveroldala egyaránt szkriptelhető Luau-val. A szerveroldali szkriptek felelnek a játéklogikáért, az adatkezelésért és a hitelesítésért, míg a kliensoldali szkriptek elsősorban a felhasználói felület megjelenítését, a bemenetek kezelését valósítják meg. A két oldal közötti kommunikáció eseményeken keresztül történik.

Mivel a platform szinte kizárólag felhasználók által írt kódokra épül, kiemelt jelentőséggel bír a biztonság. A Roblox szervereinek egyszerre több millió, potenciálisan nem megbízható szkriptet kell biztonságosan és hatékonyan futtatniuk. A Luau virtuális gépe és a Roblox engine által biztosított sandbox mechanizmusok igyekeznek megakaádlyozni, hogy rosszindulatú kódok hozzáférjen a rendszer erőforrásaihoz vagy kárt okozzanak.

A Roblox így tökéletes példája annak, hogyan tehetik a szkriptek lehetővé, hogy a játékosok ne csupán módosítsák a meglévő tartalmat, hanem maguk építsék fel azt az alapoktól kezdve. Ugyanakkor fontos biztonsági és architekturális kérdéseket is felvet, különösen a kliens–szerver szétválasztás, a szkript-elszigetelés és az egyedi szkriptnyelvek szerepével kapcsolatban.

4.8.4. Grand Theft Auto játékok

A Rockstar Games Grand Theft Auto (GTA) sorozata a nyílt világú játékok egyik legjelentősebb mérföldköve. A fejlesztés során a Rockstar kifejezetten zárt környezetben dolgozik: saját fejlesztésű motorral és belső, nem publikus szkriptelési rendszerrel, amelyet nem oszt meg a külvilággal. Ellentétben a korábban bemutatott játékokkal, a Rockstar Games nem biztosít hivatalos szkriptelési API-t vagy nyílt modding támogatást a közösség számára.

Ennek ellenére a sorozat hatalmas rajongótábora és modder közössége nem hagyta kihasználatlanul a játékokban rejlő lehetőségeket. A játékosok saját kezűleg, visszafejtési (reverse engineering) módszerekkel hoztak létre kiforrott többjátékos megoldásokat és szkriptelési rétegeket. Ezt legjobban két példán keresztül lehet szemléltetni: a Grand Theft Auto: San Andreas és a Grand Theft Auto V esetében.

A Grand Theft Auto: San Andreas esetében két jelentős projekt töltötte be az űrt: a SA-MP (San Andreas Multiplayer) és a Multi Theft Auto (MTA), ahol bárki létrehozhat saját játékszervereket. Míg a SA-MP egy egyszerűbb, C-szerű Pawn szkriptnyelvet használ, addig az MTA a sokkal rugalmasabb Lua nyelvre épít. Hosszú távon az MTA fejlődött tovább, köszönhetően a nyitottabb architektúrájának és erősebb szkriptelési lehetőségeinek. Az MTA motorja kódinjektálási és hookolási technikákkal illeszkedik az

eredeti játékba, majd a belső funkciókat egy Lua szkriptfuttató környezetben teszi elérhetővé a fejlesztők számára [15]. Az MTA szerverek erőforrásokból (resource) épülnek fel, amelyek elszigetelt, dinamikusan indítható és leállítható modulok [14].

A Grand Theft Auto V esetében a közösség válasza a FiveM keretrendszer volt. A FiveM megtartotta az MTA erőforrás-alapú architektúráját, elsősorban Lua nyelvet használ, de támogatja a JavaScript és a C# szkriptelést is [2]. A FiveM 2023-ban a Rockstar Games tulajdonába került, így napjainkra már a kiadó által engedélyezett és támogatott platformnak tekinthető. Más alternatív klienseknek (pl. RAGE Multiplayer vagy alt:V) is jelentős szerepe volt, de a FiveM nőtte ki magát a leginkább.

Ezek adták a legfőbb személyes inspirációt a jelen szakdolgozatban bemutatott szkript-architektúra megalkotásához is.

5. fejezet

Fejlesztési dokumentáció

5.1. Követelmények meghatározása

A fejlesztés elsődleges célja egy saját, általános célú játékmotor létrehozása, amely integrált szkriptelési lehetőséget biztosít. A rendszernek alkalmasnak kell lennie különböző műfajú, háromdimenziós játékok futtatására, bemutatva a szkriptnyelv és a játékmotor közötti kommunikáció hatékonyságát, valamint a szkriptelési réteg rugalmasságát.

A fejlesztés fókuszában a szoftverarchitektúra és a szkriptrendszer áll, így nem cél a legmodernebb motorok grafikai szintjének és funkcionális gazdagságának elérése. A motor feladata, hogy biztosítsa a stabil alapműködést, a megfelelő teljesítményt és a moduláris szkriptelhetőséget.

5.1.1. Funkcionális követelmények

Játékmotor követelményei:

- **Grafikus megjelenítés:** A motornak képesnek kell lennie háromdimenziós objektumok betöltésére, valamint textúrák megjelenítésére. Lehetővé kell tennie a játéktér bejárhatóságát és a kamera mozgatását belső nézetű (*First-Person View*) megközelítéssel. Támogatnia kell az alapvető megvilágítási modelleket.
- **Fizika szimulálása:** A rendszernek biztosítania kell az alapvető fizikát és a térbeli objektumok közötti ütközésérzékelést (*collision detection*).
- **Bemenetkezelés:** A felhasználói interakciók (billentyűzet, egér) valós idejű feldolgozása.
- **Hangkezelés:** Alapvető hangeffektek lejátszása.

Szkriptrendszer követelményei:

- **Integráció:** A választott szkriptnyelv értelmezőjének beágyazása a játékmotorba.
- **Kétirányú kommunikáció:** A szkripteknek képesnek kell lenniük meghívni a játékmotor natív függvényeit (például objektumok létrehozása), illetve a motornak eseményeket (például ütközés, felhasználói bemenet) kell tudnia jelezni a szkriptek felé.

- **Logika szétválasztása:** A játékmenet szabályait a motor újrafordítása nélkül, külső szkriptfájlokban, a motor magjától elkülönítve kell megvalósítani.
- **Rugalmasság:** A rendszer az eszközkészletével biztosítson lehetőséget különféle műfajú játékok prototípusának elkészítésére.

5.1.2. Nem-funkcionális követelmények

- **Teljesítmény:** A játéknak stabil képkockasebességgel (minimum 60 FPS) kell futnia közepes hardveren is, optimalizált memóriahasználat mellett.
- **Platformfüggetlenség:** A programnak olyan technológiákra kell épülnie, amelyek minimalizálják az operációs rendszerekhez kötött függőségeket, lehetővé téve a hordozhatóságot.
- **Bővíthetőség:** A rendszer architektúrájának támogatnia kell új játékelemek vagy logikák hozzáadását kizárólag szkriptírással, a motor C forráskódjának módosítása és újrafordítása nélkül.
- **Stabilitás:** A rendszernek robusztusnak kell lennie a szkriptoldali hibákkal szemben. Egy hibásan megírt szkript nem okozhatja a játékmotor összeomlását.
- **Megfelelőség:** A forráskódnak követnie kell a választott programozási szabványokat. A kódolási stílusnak egységesnek kell lennie.
- **Biztonság:** A szkriptkörnyezetnek izoláltan kell működnie. A szkriptek nem hajthatnak végre veszélyes műveleteket az operációs rendszer szintjén, kizárólag a motor által biztosított API-n keresztül kommunikálhatnak.
- **Karbantarthatóság:** A kódnak moduláris felépítésűnek, jól dokumentáltnak és átláthatónak kell lennie.

5.2. Tervezés

A fejlesztés megkezdése előtt elengedhetetlen a játékmotor szoftverarchitektúrájának és logikai felépítésének megtervezése. Mivel a cél egy robusztus, mégis könnyen bővíthető rendszer létrehozása, a tervezés során a modularitás és a felelősségi körök szétválasztásának elve (*Separation of Concerns*) jelentik a legfőbb irányelveket [22].

Kiemelt figyelmet kell fordítani a projektmenedzsmentre, a rendelkezésre álló fejlesztési idő és a határidők optimális beosztására is. A fejlesztési folyamat átláthatósága és strukturálása érdekében a projekt egymásra épülő, egyedi határidőkkel rendelkező mérföldkövekre való bontása garantálja, hogy a komplex, átfogó probléma kisebb, jól menedzselhető és mérhető részfeladatokra tagolódjon.

A projekt fejlesztési életciklusa három jól elkülöníthető fő szakaszra oszlik:

1. A játékmotor megírása.
2. A C-Lua API és a keretrendszer kiépítése.
3. A tényleges szkriptelés, amelynek során játszható prototípusok készülnek el.

5.2.1. Időbeosztás és erőforrás-tervezés

A projekt sikeres megvalósítása érdekében az előzetes tervezés során fel kell mérni az egyes fejlesztési szakaszok várható munka- és időigényét. A feladatok komplexitása és a szükséges kutatómunka alapján a rendelkezésre álló fejlesztési idő a következő becsült arányok szerint oszlik meg:

- **A játékmotor megírása (kb. 50%):** Bár a dolgozat fő témája a szkriptelési keretrendszer bemutatása, a fejlesztési idő jelentős részét maga az alapmotor kialakítása emésztheti fel. Ez a fázis intenzív kutatómunkát, valamint összetett matematikai és alacsony szintű programozási problémák megoldását igényli. Bár arányaiban ez a legidőigényesebb feladat, egy stabil játékmotor kiépítése elengedhetetlen, hiszen ez adja a biztonságos és hatékony szkriptelés alapját.

- **A C-Lua API és a keretrendszer kiépítése (kb. 30-35%):** A második leginkább időigényes szakasz a két eltérő környezet, az alacsony szintű C és a magas szintű Lua összekötése. Ide tartozik még olyan magas szintű játékelemek implementálása, amelyek a játékmotor által nyújtott funkciók mellett szintén a szkriptelés eszköztárához tartoznak.

- **A szkriptelés és prototípuskészítés (kb. 15-20%):** A fejlesztés utolsó szakasza szinte teljes egészében tiszta, kreatív munkává válik, amely az addigi erőfeszítések gyümölcseit aratja le. Ennek a szakasznak a viszonylag alacsony időigénye egyben a projekt sikerességét is jelenti: egy jól megtervezett motor és egy hatékony API birtokában a tényleges játékmenet, a menürendszer és a vizuális elemek kialakítása a Lua nyelven rendkívül gyorssá, dinamikussá és kényelmessé válik.

5.2.2. A játékmotor fejlesztési mérföldkövei

Mivel a játékmotor elkészítése a projekt leginkább időigényes része, érdemes tovább bontani a hatékonyabb előrehaladás és a fokozatos tesztelhetőség érdekében. A motor fejlesztése két kisebb, egymásra épülő fázisra bontható:

- **1. fázis: Alapvető megjelenítés és navigáció:** Ennek a szakasznak a célja egy minimálisan interaktív, leginkább megjelenítésre alkalmas grafikai szoftver megalkotása. Ide tartozik az ablakkezelés, a háromdimenziós objektumok megjelenítése és a fénykezelés. Szintén ekkor készül el a kamera alapvető mozgásának logikája. A fázis végére a virtuális tér vizuálisan megjelenik és bejárhatóvá válik.

- **2. fázis: Kiterjesztett motorfunkciók és interaktivitás:** Miután az alapvető grafikus megjelenítés stabilan működik, a második iteráció feladata az egyszerű vizuális tér játéktérré alakítása. Ide tartozik többek között az alapvető fizika és ütközésérzékelés megvalósítása, a hangkezeléshez és a felhasználói felület megjelenítéséhez szükséges eszközök implementálása.

5.2.3. Fejlesztési módszertan

A projekt főbb szakaszainak eltérő jellege és követelményei miatt célszerű hibrid módszertant alkalmazni:

- **A játékmotor megírása:** Ez a projekt leginkább strukturált, részletesen megtervezett része, ahol egy lazább szekvenciális modellt érdemes alkalmazni. Mivel az alapmotor funkciói sok esetben szigorúan egymásra épülnek (például az ablakkezelés megelőzi a grafikai megjelenítést, amely pedig az ütközésérzékelést), a komponensek implementációja logikai sorrendben, egymás után történik. Egy tiszta, szigorúan szekvenciális fejlesztési modell alkalmazása azonban nem reális, hiszen egy játékmotor komplexitása gyakran igényelhet kisebb-nagyobb utólagos finomhangolásokat.

- **A C-Lua API és a keretrendszer kiépítése:** Ez a szakasz hidat képez a merev játékmotor és a rugalmas játéklogika között. A natív API híd kiépítése leginkább egy szekvenciális folyamat, a Lua keretrendszer eszköztárának felépítése viszont már iteratívabb formában valósul meg a tesztelések során felmerülő hiányosságok alapján.

- **A szkriptelés és prototípuskészítés:** A fejlesztés ezen a ponton egy kifejezetten gyors prototípuskészítési (*rapid prototyping*) stratégiára vált, ahol rendkívül gyorsan váltja egymást a tervezés, a fejlesztés és a tesztelés hármasa. A tényleges játékmenet és a pályák építése nem egy előre megadott, szigorú specifikáció alapján történik, hanem dinamikusan, fejlesztés közben formálódik.

5.2.4. A játékmotor funkcionális terve

A játékmotor feladata, hogy elvégezze a hardverközeli és leginkább számításigényes feladatokat. A motornak támogatnia kell a számítógépi grafika modernebb megközelítéseit, de határt kell szabnia: nem célja egy csúcstechnológiás (AAA) játék felépítése. Ez összhangban van a meghatározott követelményekkel és a rendelkezésre álló idővel. A következő funkciók megvalósítása tervezett:

- **Ablakkezelés:** Az operációs rendszertől független ablak létrehozása.

- **Bemenetkezelés:** A felhasználói interakciók (billentyűzetleütések, egérmozgás és gombnyomások) valós idejű feldolgozása.

- **Textúrázott objektumok megjelenítése:** Térbeli modellek és a hozzájuk tartozó kétdimenziós textúrák kezelése, grafikus csővezetéken keresztül történő megjelenítése.

- **Fénykezelés:** Egy valósághű világítási modell implementálása.

- **Kamerakezelés, irányítás:** Egy belső nézetű (*First-Person*) kamera megalkotása, amely lehetővé teszi a játéktér bejárását.

- **Fizika és ütközésdetektálás:** A gravitáció szimulálása és az objektumokkal való ütközések és egyéb interakciók detektálása.

- **Domborzatgenerálás:** Kétdimenziós, szürkeárnyalatos magasságtérképek alapján domborzatok felépítése.

- **Részecskerendszerek:** Olyan vizuális effektek megjelenítésére alkalmas, mint a tűz vagy a füst.

- **Kétdimenziós grafika:** A felhasználói felület elengedhetetlen része, a legalapvetőbb elemek a téglalapok, képek és szövegek.
- **Hangkezelés:** Két- és háromdimenziós hangok lejátszása.

5.2.5. A szkriptelési keretrendszer terve

A keretrendszer célja, hogy elrejtse a C motor alacsony szintű komplexitását, és egy egyszerű API-t biztosítson a fejlesztők számára. A tervezett főbb komponensek a következők:

- **Erőforrás-kezelés (*Resource Management*):** A játék elemeinek moduláris, csomagszerű kezelése. A rendszernek képesnek kell lennie a csomagok egymástól elszigetelt, független futtatására, indítására és leállítására.
- **Eseményvezérelt logika:** Egy központi eseménykezelő rendszer kiépítése, ahol a szkriptek feliratkozhatnak a játékmotor eseményeire.
- **Magas szintű játékelemek:** Gyakori játékfejlesztési elemek létrehozása, melyek logikája nem a motorba van beégetve, például felvehető tárgyak, időzítők.

5.2.6. Tervezett prototípusok

A játékmotor és a keretrendszer rugalmasságának, valamint az API hatékonyságának demonstrálására két eltérő műfajú, játszható prototípus elkészítése a cél:

- **Platformer prototípus:** A cél egy ügyességi, ugrálós játék prototípusának megvalósítása, ahol a fő cél egy adott pontba való eljutás különböző statikus vagy mozgó akadályokon keresztül. További játékelemek lehetnek gyűjthető tárgyak (például érmék), korlátozott idő a teljesítésre, képességek feloldása (például gyorsabb futás, magasabb ugrás).
- **Túlélő (*Survival*) prototípus:** Egy túlélőjáték alapvető funkcióit megvalósító prototípus. Ide tartoznak különböző, a játékost jellemző statisztikák, mint az életerő, éhség, testhőmérséklet. Fontos funkció a környezettel való interakció, például favágás, kövek gyűjtése, ételszerzés. Fontos eleme az ilyen játékoknak a barkácsolás és az építés is. Klasszikus példa egy tábortűz lehelyezése a megszerzett alapanyagokból. Mindezek vizuális megjelenítéséhez elengedhetetlen egy átlátható felhasználói felület.

5.3. A fejlesztőkörnyezet és technológiák bemutatása

5.3.1. C programozási nyelv

A játékmotor megírására a C programozási nyelv került kiválasztásra. A nyelv alacsony szintű, hardverközeli jellege hatékony memóriakezelést tesz lehetővé, és garantálja a motor gyors és megbízható működését, de ugyanakkor nagyobb felelősséget is hárít a programozóra.

Napjainkban a tisztán C nyelven írt játékmotorok kevésbé elterjedtek, mert hiányoznak a nyelvből a magasabb szintű, objektumorientált nyelvek (például C#, C++) által nyújtott eszközök, többek között az osztályok vagy az öröklődés.

Bár a C nem objektumorientált nyelv, az objektumorientált programozás egy tervezési paradigma, nem pedig szigorú nyelvi adottság, így az elvei a struktúrák, mutatók használatával és tudatos tervezési döntésekkel kiválóan alkalmazhatók [7].

A motor viszonylag kis mérete és teljesítménykritikus jellege miatt a C nyelv megfelel a kitűzött célok megvalósításához. Ezt alátámasztja az is, hogy a két legelterjedtebb modern játékmotor, az Unreal Engine és a Unity szintén C-ből kinőtt nyelveken (C++ és C#) alapul [7].

5.3.2. Lua szkriptnyelv

A Lua egy rendkívül kisméretű, nyílt forráskódú, platformfüggetlen, általában beágyazott rendszerekben használt szkriptnyelv, melynek fejlesztői kifejezetten fontosnak tartják az együttműködést a C nyelven készült programokkal, sőt, maga a Lua is C-ben készült [8].

Szintaktikája jelentősen egyszerűbb, mint például a C nyelvnek, ezért a játékmechanikák és a magas szintű logika implementálása sokkal gyorsabb és átláthatóbb, ami lehetővé teszi, hogy a fejlesztő az alacsony szintű részletek helyett kizárólag a játékélményre fókuszáljon.

E tulajdonságai miatt megfelelő választásnak bizonyult a C-ben készült játékmotorral való együttműködéshez. A Lua felel a játékmenet és a felhasználói felület kialakításáért. Mivel a Lua egy értelmezett szkriptnyelv, a játéklogika módosításához nincs szükség a motor teljes újrafordítására, ami jelentősen gyorsítja a fejlesztési iterációt.

A motor és a szkriptek közötti integráció alapját a Lua hivatalos, natív C API-ja adja, amelyen keresztül kétirányú kommunikáció valósítható meg [8]. Az adatcsere egy verem (stack) segítségével történik [8].

5.3.3. OpenGL

A háromdimenziós grafika megjelenítésére és a videókártya kezelésére az OpenGL (Open Graphics Library) nevű, platformfüggetlen API került kiválasztásra.

A fejlesztés során szempont a modernebb grafikai megoldások alkalmazása, ugyanakkor a széles körű hardveres támogatottság biztosítása. Ennek megfelelően a választás az OpenGL 3.3-as verziójú "Core Profile" specifikációjára esett. Ez a verzió széles körben elterjedt szabványnak tekinthető, mivel támogatja a teljes mértékben programozható shadereket, valamint a 2010 után gyártott videókártyák túlnyomó többsége kompatibilis vele [23].

A megjelenítési folyamat során a csúcspontok (vertexek) térbeli transzformációját, a képpontok textúrázását, valamint a megvilágítási modellek alkalmazását GLSL (OpenGL Shading Language) nyelven írt programok végzik közvetlenül a videókártyán [19].

Felmerülhet a kérdés, hogy miért nem modernebb API, például a DirectX vagy Vulkan került kiválasztásra. Ezek az API-k ugyan nagyobb teljesítményt és precízebb hardveres kontrollt tesznek lehetővé, azonban jelentősen komplexebbek fejlesztési szempontból is, ami az aktuális projekt esetén szükségtelen terhet jelentene. A DirectX kizárólag Windows platformon elérhető, a Vulkan pedig bár platformfüggetlen, jóval

alacsonyabb szintű és összetettebb annál, mint amit ez a projekt indokol. Ezeket figyelembe véve az OpenGL bizonyult a feladat szempontjából a legmegfelelőbbnek.

5.3.4. SDL

Az OpenGL önmagában kizárólag grafikus API, és nem biztosít beépített eszközöket ablakok létrehozására vagy a felhasználói bemenetek kezelésére. Ezt az űrt tölti be az SDL2 (Simple DirectMedia Layer) könyvtár.

Az SDL elsődleges feladata az operációs rendszertől független ablak létrehozása, valamint a grafikus rendereléshez elengedhetetlen OpenGL kontextus inicializálása [20]. Ennek köszönhetően a játékmotor képes egységes módon elindulni és működni különböző operációs rendszereken.

Mindemellett a könyvtár végzi a játékos interakcióinak, főként az egér és a billentyűzet bemeneteinek feldolgozását [20]. A projekt továbbá támaszkodik a könyvtár hivatalos kiegészítő moduljaira is: a textúrák betöltését az SDL_image, míg a felhasználói felület szövegeinek megjelenítését az SDL_ttf modul végzi, a hangkezelésben pedig az SDL_mixer biztosít segítséget.

5.3.5. További felhasznált könyvtárak

A komplexebb keretrendszerek mellett a játékmotor több kisebb, egy-egy specifikus feladatot ellátó nyílt forráskódú C könyvtárat is integrál:

- **GLAD:** Mivel az OpenGL csupán egy specifikáció, a grafikus függvények memóriacímeit futásidőben kell lekérdezni a videókártya illesztőprogramjától [23]. A GLAD automatikusan beazonosítja és elérhetővé teszi a C kód számára az OpenGL 3.3-as specifikációjához tartozó függvényeket.

- **CGLM (C Graphics Library Math):** A háromdimenziós grafika jelentős mennyiségű vektor- és mátrixműveletet igényel, amelyekhez a C nyelv nem biztosít beépített adattípusokat. A játékmotor ezen a matematikai könyvtáron keresztül végzi a különféle transzformációkat.

- **fast_obj:** Egy rendkívül gyors és kisméretű C nyelvű könyvtár, amely a 3D modellek (jellemzően Wavefront .obj formátumú fájlok) feldolgozásáért felel. Segítségével a motor hatékonyan olvassa be a modellek csúcspontjait, normálvektorait és textúrakoordinátáit a memóriába, ahonnan azok az OpenGL számára feldolgozható formában továbbíthatók a videókártyára.

5.3.6. Fordítási környezet és eszközök

A forráskód gépi kóddá alakításához a GCC fordítóprogram került alkalmazásra. Mivel a program nagyszámú forrásfájlból áll, és számos külső könyvtárat integrál, a gyors és hibamentes fordítás érdekében egy Makefile segítségével egyetlen parancs kiadásával elvégezhetők a fordítási és linkelési folyamatok.

5.4. A játékmotor implementálása

5.4.1. Az alkalmazás felépítése és működési ciklusa

A program belépési pontja a `main.c` fájl, amely a program teljes életciklusát vezérli: az inicializálástól kezdve a futási cikluson át egészen az erőforrások felszabadításáig.

Az alkalmazást és annak `App` struktúráját az `app.c` `init_app` függvénye inicializálja. Az inicializálás során az SDL könyvtár beállítja az OpenGL 3.3 Core Profile kontextust, majd létrehozza az ablakot az aktuális kijelző felbontásával megegyező méretben. Ezt követően inicializálódnak a külső könyvtárak (`SDL_image`, `SDL_ttf`, `SDL_mixer`), a GLAD betölti az OpenGL függvényeket, majd betöltődnek a shaderek, a kamera, a színtér, a szkriptrendszer, a felhasználói felület, a részecskerendszer és a tereprendszer.

Bár a cél a háromdimenziós megjelenítés, nyilvánvaló, hogy a kijelző, ami megjeleníti a grafikát, csupán kétdimenziós. Emiatt az egyik legfontosabb lépés, hogy a háromdimenziós térben meghatározott entitásokat leképezzük kétdimenziósra úgy, hogy valósághű megjelenést nyújtsanak. Ennek a valósághű megjelenésnek az egyik legfontosabb tulajdonsága a perspektíva, amely során az objektumok távolságát a méretük érzékelteti. A közelebb lévők nagyobbak, a távolabb lévők kisebbek. Ezt perspektivikus projekciónak nevezik [10].

A perspektivikus projekció végrehajtható a `cglm` által nyújtott `glm_perspective` függvénnyel, ami bemenetként egy vertikális látószöget ($FOVy$), egy képarányt (az ablak szélességének és magasságának hányadosa), illetve egy úgynevezett *near plane* és *far plane* értéket vár [10]. Ezen paraméterek helyes megválasztásához fontos megérteni, hogy a függvény pontosan mit állít be. A látóteret egy csonkagúlaként (*frustum*) határozzuk meg, amelynek csúcsa a kamera pozíciójában helyezkedik el [10]. Az alsó és felső lapok síkját a látószög, a bal és jobb oldalt pedig a képarány határozza meg [10]. A *near plane* és a *far plane* határozza meg a gúla két végét, így kapjuk a csonkagúlát. A *far plane* megadja, hogy legfeljebb milyen távolságban lévő elemek rajzolódjanak ki, míg a *near plane* azt a minimális távolságot definiálja, amelynél közelebb lévő objektumok már nem kerülnek kirajzolásra [10]. Más szavakkal a látótér a *near plane* és *far plane* közé eső részt jelenti, az ennél közelebbi vagy távolabbi objektumok nem kerülnek megjelenítésre. Ezekből a paraméterekből állítja elő a `glm_perspective` függvény a projekciós mátrixot és eltárolja az `App` struktúrában.

A kétdimenziós felhasználói felület megjelenítésekor más megközelítésre van szükség. Míg a háromdimenziós tér megjelenítéséhez elengedhetetlen a perspektíva, a felhasználói felület kirajzolásakor erre nincs szükség. Ezt az úgynevezett ortogonális vetítés teszi lehetővé, amelyet a motor a `glm_ortho` függvénnyel valósít meg. Ennek a függvénynek meg kell adni a bal, jobb, alsó és felső határokat, valamint a `nearZ` és `farZ` értékeket [10]. Bár az OpenGL alapértelmezés szerint a képernyő bal alsó sarkát tekinti origónak [23], a felhasználói felület elemeinek pozicionálását jelentősen leegyszerűsíti, ha a képernyő bal felső sarkát definiáljuk a $(0, 0)$ pontként. Ezt a motor úgy éri el, hogy a vetítés bal és felső határait 0-ra, míg a jobb oldali értéket az ablak szélességére, az alsó határt pedig az ablak magasságára állítja be. Ahogy említésre került, ortogonális megjelenítés esetén nincs perspektíva, mégis meg kell adni a `nearZ` és `farZ` értékeit. Ennek az az oka, hogy az OpenGL alapvetően térben működik, így az ortogonális vetítés valójában nem egy kétdimenziós téglalapot, hanem egy téglatestet hoz létre [10]. Ennek előnye, hogy a felhasználói felület elemei rétegezhetők, és meghatározott sorrendben rajzolhatók ki.

A fő ciklus minden iterációban három lépést hajt végre: az események feldolgozását

(`handle_app_events`), az állapotok frissítését (`update_app`), valamint a kirajzolást (`render_app`) [7]. A ciklus addig fut, amíg az `app.is_running` értéke igaz. A frissítési és renderelési sebesség hardverfüggetlenségét a `delta_time` (a két képkocka között eltelt idő) számítása és alkalmazása garantálja [7].

Az eseménykezelés az SDL által nyújtott eseményrendszeren keresztül valósul meg. Az SDL eseményrendszere lehetővé teszi a felhasználói interakciók, például billentyűleütések, egérkattintások és kurzormozgások detektálását [20]. A `handle_app_events` függvény feladata ezen események feldolgozása. A feldolgozás során a motor egy ciklus segítségével lekérdezi az eseményeket az `SDL_PollEvent` függvénnyel, majd azok típusától függően különböző műveleteket hajt végre [20]. Például a billentyűzet és egér események hatására módosulhat a kamera állapota vagy a felhasználói felület, míg az `SDL_QUIT` esemény hatására az alkalmazás leáll.

A program leállásakor a `destroy_app` függvény gondoskodik az összes lefoglalt erőforrás, a memória, a textúrák, a shaderek és az alrendszerek felszabadításáról.

5.4.2. A színtér

A játékmotorban a színtér (*scene*) felelős a virtuális világ nyilvántartásáért, frissítéséért és kezeléséért. Ez fogja össze a játékost körülvevő összes elemet: az objektumokat és textúrákat, a környezeti tényezőket (például a domborzatot és az égboltot), a részecskerendszereket, valamint a megvilágítást.

A színtér alapvető építőkövei az entitások (*entity*), amelyek a térben elhelyezett objektumok. A motor az entitásokat egy előre lefoglalt, fix méretű statikus tömbben tárolja. Minden egyes entitás önálló állapottal rendelkezik, amely magában foglalja a térbeli transzformációs adatait (pozíció, forgatás, skálázás), az anyagjellemzőit (például a csillogás mértékét), a hozzárendelt 3D modell és textúra azonosítóját, valamint mozgó entitások esetén a frissítésekhez szükséges pozíció- és rotációváltozásokat (delta értékeket). A szoftver ezen felül támogatja az időzített, célvezérelt mozgásokat is: az entitások képesek lineáris interpolációval eljutni egy megadott kezdőpontból egy célpontba egy előre meghatározott időintervallum alatt [10].

A frissítési fázisban a motor végigiterál az entitásokon, kiszámítva az objektumok adott képkockára eső elmozdulását és új pozícióját. Ezt követi a renderelési fázis, ahol a rendszer a már frissített transzformációs adatokat modellmátrixokká alakítja, majd a vizuális beállításokkal együtt továbbítja azokat a videókártya grafikus csővezetéke felé.

5.4.3. Kamerakezelés és irányítás

Egy interaktív grafikus programban elengedhetetlen, hogy a virtuális tér bejárható legyen. Ez többféle megközelítéssel is megvalósítható. A számítógépes játékok és grafikus motorok története során számos klasszikus kameranézet alakult ki, amelyek alapvetően meghatározzák a felhasználói élményt és a szoftver jellegét.

A leggyakoribb típusok közé tartozik a külső nézet (*Third-Person View*), ahol a kamera a térben elhelyezett karaktert a háta mögül vagy felülről követi, ezáltal kiváló rálátást biztosít a környezetre és filmszerű élményt nyújt. Szintén elterjedt az izometrikus vagy felülnézeti (*Top-Down*) kamera, amelyet elsősorban stratégiai játékok alkalmaznak, egy rögzített szögből átfogó képet nyújtva a játéktérről. Fejlesztői környezetekben gyakori a szabad kamera (*Free Camera* vagy *Spectator*), amely fizikai korlátok és gravitáció nélkül, szabadon mozgatható a térben. A jelen projekt azonban

a leginkább immerzív megoldást, a belső nézetes (*First-Person View*) rendszert valósítja meg. Ebben a felállásban a virtuális kamera közvetlenül a játékos által irányított karakter szemmagasságában helyezkedik el, ezáltal a felhasználó a saját szemszögéből élheti át a virtuális tér eseményeit.

Az irányítási bemeneteket az alkalmazás fő ciklusa dolgozza fel. Az egér elmozdulása közvetlenül a kamera forgatási szögeit módosítja. Annak érdekében, hogy a nézet ne forduljon át önmagán, a vertikális forgatás -89 és 89 fok közé van korlátozva. A billentyűzet gombjai (`WASD`) a haladási vektorok irányát állítják be, míg a módosító billentyűk (`Shift`, `Ctrl`, `Space`) az állapotjelzőket (futás, guggolás, ugrás) aktiválják.

A tényleges elmozdulás kiszámítása a renderelési ciklustól függetlenül, a képkockák között eltelt idő (*delta time*) alapján történik. Ez garantálja, hogy a mozgás sebessége egyenletes maradjon, függetlenül a számítógép teljesítményétől [7]. A vízszintes haladást az aktuális állapot is befolyásolja: futás esetén a rendszer megkétszerezi, míg guggolásnál megfelezi az alapértelmezett sebességet. A függőleges tengelyen (Z) a kamerát fizikai szimuláció éri: a gravitáció folyamatosan lefelé gyorsítja a karaktert, amíg talajt nem ér.

A számítógépes grafikában azonban a kamera csupán egy elméleti absztrakció. Matematikai szempontból a kamera két alapvető komponensből épül fel: egy térbeli pozícióból (c_x, c_y és c_z koordináták), valamint a rotációból (forgatási szögekből). A gyakorlatban például a kamera előremozdításakor valójában nem a kamera mozog, hanem a teljes színteret (beleértve az összes entitást, domborzatot és fényforrást) toljuk el a kamerával pontosan ellentétes irányba, tehát hátrafelé [23]. Ezt az inverz transzformációt a 3D grafikában nézeti mátrixnak (*view matrix*) nevezzük.

Ennek matematikai háttere legkönnyebben a kétdimenziós tér geometriájából vezethető le. A hagyományos lineáris transzformációk során a forgatásokat mátrixszorzással írjuk le, amely azonban az origót mindig helyben hagyja, így az eltolás (transzláció) önmagában csak összeadással lenne megvalósítható [10]. Azért, hogy az eltolást is elvégezhessük mátrixszorzással, bevezethetünk egy plusz dimenziót (az úgynevezett homogén koordinátát), melynek értéke 1 [10].

Ezen mátrixos transzformációk matematikai és programozási kiindulópontja az úgynevezett egységmátrix (identitásmátrix, I) [10]. Ez a transzformációk alapállapota, amely önmagában semmilyen eltolást vagy forgatást nem tartalmaz. Főátlójában egyesek, minden más pozícióban pedig nullák találhatók. A háromdimenziós grafikában használt 4×4-es egységmátrix a következőképpen néz ki:

$$I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A gyakorlatban minden térbeli mozgás ebből az alapállapotból épül fel. A 2D-s térben a pontokat egy harmadik (homogén) koordinátával egészítjük ki, a transzformációkat pedig 3×3-as mátrixok írják le [10]. A síkban történő forgatás csupán egyetlen tengely (az origó) körül történhet egy φ szöggel, melynek transzformációs mátrixa az alábbi [10]:

$$R_{2D} = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Amikor átlépünk a háromdimenziós térbe, a koordinátarendszer és a homogén koordináta együtt már négy dimenziót alkot, így a transzformációk 4×4-es mátrixokat igényelnek. A térben történő forgatás már három független tengely (X, Y, Z) körül is történhet, amelyeket az α, β és γ Euler-szögek írnak le [10]. A megfelelő tengelyek körüli forgatási mátrixok a $2D$-s mátrix kiterjesztései:

Az X tengely körüli forgatás mátrixa:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Az Y tengely körüli forgatás mátrixa:

$$R_y = \begin{bmatrix} \cos\beta & 0 & \sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A Z tengely körüli forgatás mátrixa:

$$R_z = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 & 0 \\ \sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Mivel a nézeti mátrix (V) valójában a kamera saját transzformációjának inverze, a végső mátrix ezen forgatások transzponáltjának (R^T) és a negált eltolásnak (T^{-1}) a szorzatából áll elő [10]:

$$V = R^T \cdot T^{-1}$$

A képlet inverz eltolási mátrixa (T^{-1}) szintén a 4×4-es egységmátrixra épül, amelynek utolsó oszlopába a kamera aktuális térbeli koordinátái negálva kerülnek be [10]. Ez biztosítja, hogy a világ a kamera pozíciójával ellentétes irányba tolódjon el:

$$T^{-1} = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 1 & -c_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A képlet forgatási komponensét a három tengely körüli mátrix (R_x, R_y, R_z) megfelelő sorrendben vett szorzata adja. Az ortogonális (tisztán forgatást leíró) mátrixok inverze pontosan megegyezik a transzponáltjukkal [10]. A transzponálással kapott inverz forgatási mátrix (R^T) elemeit egyszerűsítve r_{ij} változókkal jelöljük. Ennek a mátrixnak a sorai valójában a kamera lokális koordináta-rendszerében definiált irányvektorokat (előre, jobbra, fel) jelentik:

$$R^T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A két mátrix összeszorzása szorzása után előállt végső, 4×4-es nézeti mátrix jobb szélső oszlopa már a forgatás és az inverz eltolás skaláris szorzatát fogja tartalmazni [10]. Ez a gyakorlatban azt a távolságot számítja ki, amennyivel a színteret el kell tolni az adott nézési iránynak megfelelően:

$$V = \begin{bmatrix} r_{11} & r_{12} & r_{13} & -(r_{11}c_x + r_{12}c_y + r_{13}c_z) \\ r_{21} & r_{22} & r_{23} & -(r_{21}c_x + r_{22}c_y + r_{23}c_z) \\ r_{31} & r_{32} & r_{33} & -(r_{31}c_x + r_{32}c_y + r_{33}c_z) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A játékmotorban ezeket a komplex lineáris algebrai műveleteket nem szükséges manuálisan elvégezni. A kamerakezelő modulban a `cglm` matematikai könyvtár a képkockák renderelése előtt mindig egy egységmátrixszal inicializálja a nézetet, majd beépített eltoló és forgató függvényei építik fel a végső mátrixot a kamera paramétereiből. A kiszámított mátrix ezt követően változóként kerül átadásra a shaderek felé, ahol a *vertex shader* végrehajtja a tényleges térbeli vetítést a csúcspontokon.

A következő kódrészlet szemlélteti, hogy a `cglm` könyvtár segítségével a teljes folyamat néhány függvényhívással megvalósítható. Bár az elmélet három független forgatást (bólintás, fordulás, dőlés) ír le, a *First-Person* kamera esetében a karakter oldalra dőlése (*roll*) szándékosan nincs implementálva, így ez a mátrixszorzás kimarad az optimalizáció és a stabil horizont megtartása érdekében. Továbbá fontos megjegyezni a második, fix -90 fokos X-tengely körüli forgatást is: mivel a motor fizikai rendszere "*Z-up*" (a Z tengely mutat felfelé) koordináta-rendszert használ, míg az OpenGL az "*Y-up*" rendszert várja el [23], ezzel a korrekciós forgatással hangoljuk össze a játéktér vertikális tengelyét a grafikus motoréval.

Programkód 5.1. A nézeti mátrix (view matrix) kiszámítása a cglm könyvtárral

```
glm_mat4_identity(camera->view_matrix);

glm_rotate(camera->view_matrix,
-glm_rad(camera->rotation.x),
(float[]){1.0f, 0.0f, 0.0f});

glm_rotate(camera->view_matrix,
glm_rad(-90.0f),
(float[]){1.0f, 0.0f, 0.0f});
```

```
glm_rotate(camera->view_matrix,
-glm_rad(camera->rotation.z + 90.0f),
(float[]){0.0f, 0.0f, 1.0f});

glm_translate(camera->view_matrix,
(float[]){-camera->position.x,
        -camera->position.y,
        -camera->position.z});
```

5.4.4. Fények

A valósághű, háromdimenziós megjelenítés alapvető feltétele a megfelelő megvilágítás. Fényhatások nélkül a térbeli objektumok alakja és mélysége nem érzékelhető, mivel a felületek között nem alakul ki látható különbség. Az objektumok megjelenését nem csupán saját tulajdonságaik határozzák meg, hanem a színtér megvilágítása is. A közvetlenül megvilágított felületek világosabbnak látszanak, míg a takarásban lévő részek sötétebbek. A fényforrások és az általuk keltett árnyalatok teszik lehetővé a térbeli formák értelmezését, ezért ezek modellezése lényeges része a játékfejlesztésnek.

A játékmotor a széles körben elterjedt Phong-féle megvilágítási modellt implementálja [23]. Ez a modell három különböző komponensből áll:

- **Környezeti fény (*ambient*):** A környezeti fény mindenhol jelen van a színtérben. Emiatt nem rendelkezik pozícióval, csak színkomponensekkel (piros, zöld és kék) jellemezhető. Feladata az alapvető megvilágítás biztosítása, így az objektumok akkor sem teljesen feketén jelennek meg, ha nem éri őket más fényforrás [23].

- **Szórt fény (*diffuse*):** A szórt fény a fényforrásból érkező közvetlen megvilágítást modellezi, és jelentős szerepet játszik az objektumok térbeli megjelenésének kialakításában. A fény erősségét befolyásolja a fényforrás pozíciója, valamint az, hogy a fény milyen szögben éri a felületet. Azok a felületek, amelyekre a fény közvetlenül esik, világosabbnak látszanak, míg a nagyobb szöget bezáró felületek sötétebbek [23].

- **Tükröződő fény (*specular*):** A tükröződő vagy spekuláris fény a sima, fényes felületeken (például fémeken, üvegen) megjelenő, csillogó fényhatásokat modellezi. A szórt fénnyel ellentétben ennek érzékelése nem kizárólag a fényforrás pozíciójától függ, hanem nagymértékben függ a kamera helyzetétől is. A tükröződés mértékét és élességét a felületek anyagjellemzői határozzák meg [23]. A játékmotorban minden entitás saját tükröződési színkomponensekkel, valamint egy csillogási (*shininess*) értékkel rendelkezik. A `shininess` paraméter a felületen megjelenő tükröződő fényfolt méretet szabályozza. Minél magasabb ez az érték, a felület annál simábbnak tűnik, így a csillogás pontszerűbb lesz, míg alacsony értéknél a teljes felületen látható tompább csillogás [23].

A három komponens kombinációja adja meg az objektumok valósághű megvilágítását a színtérben.

A játékmotor különböző típusú fényforrások létrehozását teszi lehetővé:

- **Környezeti fény (*ambient light*):** A fentebb leírtaknak megfelelően a színkomponenseken (piros, zöld, kék) kívül nincs más jellemzője.

- **Irányított fény (*directional light*):** Pozícióval nem, csak iránnyal rendelkező fénytípus, amelynek erőssége nem tompul a távolsággal [23]. Gyakran ez a fő fényforrás, feladata a színtér alapvető bevilágítása, ez például a napsugarak szimulálását jelentheti.

- **Pontfény (*point light*):** Pozícióval rendelkezik, irány azonban nem jellemzi, a tér minden irányába egyenletesen sugároz. Intenzitása a fényforrástól való távolság függvényében csökken. Ezt a fény tompításának (*attenuation*) nevezik [23]. A játékmotor a tompítás mértékét az alábbi képlettel számítja ki, közvetlenül a programban implementált változókat használva [23]:

  $$\text{attenuation} = \frac{1}{\texttt{constant} + \texttt{linear} \cdot d + \texttt{quadratic} \cdot d^2}$$

  Ahol d a megvilágított pont és a fényforrás közötti távolságot jelöli. A képletben az állandó (`constant`) tényező (amely értéke jellemzően 1) megakadályozza, hogy a nevező értéke nulla legyen. [23]. A lineáris (`linear`) és másodfokú (`quadratic`) tényezők pedig a megvilágítás valósághű, távolságfüggő csökkenéséért felelnek, ezáltal a fényerő eleinte lassan, majd a távolság növekedésével egyre gyorsabban esik vissza [23]. A pontfények ideálisak például lámpák, fénylő objektumok vagy tűzeffektek szimulálására. A motorban minden pontfény egy pozíciót, színkomponenst (piros, zöld, kék) és az előbbi tompítási paramétereket kapja meg.

- **Spot lámpa (*spotlight*):** A spot lámpák irányított fényforrások, amelyek egy adott pontból, meghatározott irányba, kúp alakban sugároznak [23]. Szintén rendelkeznek tompítási együtthatókkal (`constant`, `linear`, `quadratic`), amelyek a fény intenzitásának csökkenését szabályozzák a távolsággal. A motor a fényszórás kúpjának méretét és a peremének lágyságát egy belső (`cutOff`) és egy külső vágószög (`outerCutOff`) segítségével definiálja. A `cutOff` értéke határozza meg azt a szöget, amelyen belül a fény teljes intenzitással érvényesül, míg az `outerCutOff` azt a határt jelöli, amelyen túl a fény már nem látható. A két szög közötti tartományban a fény intenzitása fokozatosan csökken [23]. Ezek a fények jól alkalmazhatók például zseblámpák modellezésére.

Ezekkel a fénytípusokkal kielégítő módon szimulálhatók a valóságos fények.

5.4.5. Objektum- és textúrakezelés

A grafikus alkalmazások két legfontosabb építőeleme a térbeli geometria (objektumok vagy modellek) és a rajtuk megjelenő mintázatok (textúrák).

Egy 3D objektum a virtuális térben elhelyezkedő formát határozza meg, amely lehet egy egyszerű mértani test, de akár egy komplex épület is. Matematikailag ezek a modellek térbeli pontok, úgynevezett csúcspontok (*vertex*ek) halmazából állnak, amelyeket élek kötnek össze, így sokszögeket (*polygon*okat) alkotnak [10]. Mivel a modern videókártyák grafikus csővezetéke háromszögek feldolgozására van optimalizálva, végső soron minden térbeli testet háromszögekből álló hálóvá (*triangle mesh*) kell konvertálni [19]. A projektben alkalmazott modellek már eleve háromszögesítve kerülnek exportálásra a

modellező szoftverből, így a programnak nem kell erőforrást fordítania a komplexebb poligonok átalakítására.

Az objektumok adatainak tárolására és beolvasására a választás a széles körben elterjedt Wavefront `.obj` formátumra esett. Bár napjainkban léteznek kompaktabb, bináris formátumok, az `.obj` egy egyszerű, ember által is olvasható (*plain text*) struktúrával rendelkezik. Ez a felépítés megkönnyítette a beolvasó algoritmus, a `fast_obj` könyvtár integrálását.

Egy 3D objektum adatstruktúrája több, egymáshoz szorosan kapcsolódó komponensből épül fel. Egy modell meghatározásában a legalapvetőbb adatok a csúcspontok koordinátái (X, Y, Z). Továbbá minden csúcsponthoz tartozik egy normálvektor, amely a felületre merőleges irányt mutatja meg. Ez a fénykezelés során elengedhetetlen tulajdonság. Ahhoz pedig, hogy a modellt kétdimenziós képekkel lehessen bevonni, textúrakoordinátákra (U, V) is szükség van [10]. Végül meg kell határozni, hogy ezen adatokból hogyan épüljenek fel maguk a háromszögek. Ezeket a felületeket lapoknak (*face*) nevezzük, amelyek az `.obj` fájlban perjelekkel elválasztott, 1-től kezdődő index-hivatkozások (`vertex_index/texture_index/normal_index`) listájaként írják le az összeköttetéseket. Ennek szemléltetésére kiváló példa egy egyszerű kocka `.obj` fájljának részlete:

```
v -0.500000 -0.500000 0.500000
v 0.500000 -0.500000 0.500000
... (összesen 8 csúcspont)

vt 0.000000 0.000000
vt 1.000000 0.000000
... (összesen 4 textúrakoordináta)

vn 0.000000 0.000000 1.000000
vn 0.000000 1.000000 0.000000
... (összesen 6 normálvektor)

f 1/1/1 2/2/1 3/3/1
f 3/3/1 2/2/1 4/4/1
... (összesen 12 háromszög)
```

A fenti példában a `v` (*vertex*) jelöléssel kezdődő nyolc sor a kocka nyolc sarkának térbeli koordinátáit határozza meg. A `vt` (*vertex texture*) sorok a textúraillesztéshez szükséges kétdimenziós UV koordinátákat adják meg, míg a `vn` (*vertex normal*) a normálvektorokat reprezentálja [23]. A fájl legfontosabb részét az `f` (*face*) sorok adják, amelyek magukat a háromszögeket definiálják [23]. Minden `f` sor három pontot köt össze, ahol az értékek perjelekkel elválasztva (például 1/1/1) hivatkoznak a korábban definiált térbeli csúcspont, textúrakoordináta és normálvektor indexére [23]. Mivel a kocka hat négyzet alapú oldalból áll, a fájl összesen tizenkét háromszöget definiál a teljes felület lefedésére.

A modellek önmagukban még nem nyújtanak valósághű vizuális élményt, a végső látványt a textúrák alkalmazása adja meg. A textúra kétdimenziós kép, amely ráfeszül a modellre az UV-leképezés alapján [23]. Minden csúcspont rendelkezik egy (U, V) koordinátával, amely meghatározza, hogy a textúra melyik képpontja tartozik az adott csúcshoz [23].

Egy játékmotor fejlesztésénél kritikus szempont az erőforrás-kezelés hatékonysága. Ez hatványozottan igaz olyan potenciálisan nagyméretű fájlok betöltése és használata során, mint a modellek és textúrák. Éppen ezért szükséges volt egy-egy gyorsítótár (*cache*) alkalmazása a modellek és a textúrák betöltésekor egyaránt. Ez biztosítja, hogy egy modell vagy textúra ne kerülhessen be többször a memóriába. A már egyszer betöltött adatokat a program eltárolja, és újbóli betöltési kísérlet esetén először ellenőrzi, hogy az adott erőforrás már betöltésre került-e korábban. Amennyiben igen, nem olvassa be újra a fájlból, csupán visszaadja a már meglévő OpenGL azonosítót vagy memóriacímet. Ezzel elkerülhető a felesleges memóriafoglalás, és jelentősen csökken a lemezműveletek száma: ha például száz azonos modellt és textúrát helyezünk el a színtéren, a program csak a legelső alkalommal fordul a fájlrendszerhez, a továbbiakban a gyorsítótárból szolgálja ki a kérést [7].

A textúrák fájlból történő memóriába emelését az `SDL_image` modul végzi. A motor minden beolvasott képet automatikusan `RGBA32` formátumra konvertál, így biztosítva a textúrák egységes kezelését. A nyers képpontok ezt követően az OpenGL grafikus csővezetékébe kerülnek, ahol textúra objektumokká alakulnak. A motor ismétlődő mintázatot (`GL_REPEAT`) és bilineáris szűrést (`GL_LINEAR`) alkalmaz rajtuk, amely sima és folytonos megjelenést eredményez a felületeken [23]. A textúrakezelő modul emellett támogatja a speciális köbös textúrák (*cubemap*) betöltését is, amelyek a környezeti égbolt (*Skybox*) megjelenítéséhez elengedhetetlenek [23].

Ahhoz azonban, hogy a beolvasott objektumok elfoglalhassák a tényleges helyüket a virtuális térben, szükség van egy transzformációra, a modellmátrixra (*Model Matrix*). A betöltött `.obj` fájlok csúcspontjai a saját, úgynevezett lokális koordinátarendszerükben (*Local Space*) vannak megadva, jellemzően az origó körül [23]. A modellmátrix feladata, hogy ezeket a pontokat áthelyezze a globális világtérbe (*World Space*).

A modellmátrix (M) valójában három különálló geometriai transzformáció. A skálázás, a forgatás (R) és az eltolás (T) megfelelő sorrendben elvégzett szorzataként áll elő [10]:

$$M = T \cdot R \cdot S$$

A műveletek sorrendje fontos: először a modell méretét állítjuk be, majd elforgatjuk az origója körül, végül pedig eltoljuk a végleges pozíciójába [10]. A méretezést leíró S mátrix a főátlójában tartalmazza az X, Y és Z tengelyek menti nyújtási és zsugorítási tényezőket (s_x, s_y, s_z) [10]:

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A forgatást leíró R mátrix pontosan a kamerakezelésnél bemutatott tengelyenkénti forgatási mátrixok (R_x, R_y, R_z) szorzata. Az eltolást végző T mátrix pedig az identitásmátrix utolsó oszlopában tárolja a kívánt térbeli pozíciót (t_x, t_y, t_z) [10]:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5.4.6. Shaderek

A modellek és textúrák beolvasott és gyorsítótárazott adatai önmagukban még csak nyers információk a memóriában. Ahhoz, hogy ezekből a képernyőn megjelenő kép legyen, el kell juttatni őket a videókártyának, ahol a shaderek (árnyékoló programok) végzik el a tényleges feldolgozást. Ezek egy úgynevezett csővezetéken (*pipeline*) keresztül, egymás után futnak le. A korábbiakban kifejtett elméleti modellek, struktúrák és transzformációk itt érnek össze, és itt történik meg a tényleges grafikai megjelenítés.

A játékmotorban a shader fájlok beolvasását a `shader.c` modul végzi el. A folyamat során a motor elsőként beolvassa a `.vert` (*vertex*) és `.frag` (*fragment*) kiterjesztésű fájlok tartalmát a memóriába. Ezt követően az OpenGL `glCreateShader` és `glCompileShader` függvényeivel megtörténik a forráskód lefordítása a GPU számára érthető gépi kódra [23]. Amennyiben a fordítás sikeres, a két árnyékoló egy közös programmá (*shader program*) linkelődik össze a `glLinkProgram` meghívásával [19].

A motor az adatok átadásához *Vertex Buffer Object* (VBO) és *Vertex Array Object* (VAO) struktúrákat használ, amelyek segítségével a csúcspontok adatai közvetlenül a GPU memóriájába kerülnek.

A *Vertex Buffer Object* (VBO) egy memóriabuffer a videókártya memóriájában, amelyet csúcspont-adatok tárolására használunk [23]. Előnye, hogy az adatok közvetlenül a videókártya memóriájában tárolódnak, így nem szükséges folyamatosan kommunikálnia a processzornak és a videókártyának, elegendő egyszer átadni az adatokat. Mivel azonban ez a buffer csak egy nyers bájtsorozat, a videókártyának szüksége van egy leíróra, hogy tudja, melyik bájt mit jelent és hogyan kell értelmeznie azokat.

Ezt a feladatot látja el a *Vertex Array Object* (VAO), amely szintén a videókártya memóriájában található [23]. Eltárolja, hogy az egyes attribútumokhoz melyik VBO tartozik és hogyan kell olvasni belőle. Például leírja, hogy a pozíció (X, Y, Z) vagy az UV-koordináták hol kezdődnek a bufferben és mekkora lépésközzel kell olvasni őket. A tényleges értelmezés a *vertex shaderben* történik, amely az attribútumokat a saját bemeneti változóihoz rendeli.

A shader két különböző típusú bemenetet kaphat. A legfontosabb megfontolandó kérdés az, hogy az adott adat milyen gyakran változik:

- `layout (location = x) in:` Olyan adatok kerülnek tárolásra ilyen módon, amelyek a shader minden egyes lefutásakor változnak [23]. Ilyen adat a csúcspontok pozíciója, a hozzájuk tartozó normálvektor és a textúrakoordináta. Ezek a VBO-ból kerülnek betöltésre. A `location = x` értéke egy egész szám, amely meghatározza, hogy a shader melyik slotból olvassa be az adott változót [23]. A C kódban be kell állítani, hogy az egyes slotokhoz tartozó adat hol helyezkedik el a VBO-ban, és milyen típusú. Ehhez a `glEnableVertexAttribArray` és `glVertexAttribPointer` függvények alkalmazandók. Előbbi feladata az adott azonosítójú attribútum engedélyezése a videókártyán, míg utóbbi a memória tényleges szerkezetét írja le [23]. Bár a shaderben a `location` megadása nem kötelező

lépés, érdemes eszerint eljárni, mert másképp automatikusan kerülnek kiosztásra az adatok, ami nem várt viselkedést eredményezhet.

- `uniform`: Az olyan tulajdonságokat, amelyek egy objektum kirajzolása során állandóak és minden csúcspontra egyformán érvényesek, `uniform` változóként tároljuk [23]. Ezek tipikusan a transzformációs mátrixok, tehát a projekciós mátrix, a nézeti mátrix és a modellmátrix, amely magában foglalja a modell eltolását, forgatását és skálázását. Ezeket közvetlenül a CPU adja át a shadernek, két lépésben: elsőként a `glGetUniformLocation` függvénnyel le kell kérdezni az adott változó azonosítóját a shaderben [23]. Ezután a `glUniform` függvénycsaláddal történik a tényleges értékadás. Azért nem elegendő egyetlen `glUniform` függvény, mert nem támogatott a függvénytúlterhelés. Emiatt minden adattípushoz és mérethez saját függvényre van szükség: például egy egész szám átadására a `glUniform1i`, egy háromdimenziós vektoréra a `glUniform3f`, míg a transzformációs mátrixok átküldésére a `glUniformMatrix4fv` szolgál.

A pipeline két legfontosabb, programozható állomása a *vertex shader* és a *fragment shader*.

Az első állomás, a *vertex shader* feladata az MVP (*Model-View-Projection*) transzformáció elvégzése, amellyel meghatározza minden csúcspont végső pozícióját a képernyőn [23]. A shader program pontosan annyiszor fut le, ahány csúcspontja az adott modellnek van [19]. A `gl_Position` változó az OpenGL egy beépített, speciális kimeneti változója, amelynek értékadása minden *vertex shaderben* kötelező [19]. Ez a változó tárolja a csúcspont végső koordinátáit, amelyet a lokális pozíció és az MVP mátrixok szorzataként kapunk meg [23].

További kimenetek is definiálhatók az `out` kulcsszóval, amely a *fragment shader* bemeneteként fog szolgálni [23]. Továbbítandó adat a világtérbeli pozíciót tároló `FragPos`, a normálvektort továbbító `Normal` és a textúrakoordinátákat tartalmazó `TexturePos` [23].

A *vertex shader* lefutása után egy fontos hardveres lépés, a raszterizálás (*rasterization*) következik [23]. Ekkor a videókártya a háromszögeket felbontja egyedi képpontokra (fragmensekre) [23].

A pipeline következő állomása a *fragment shader*, amely a raszterizált adatokat bemeneti (`in`) változókként fogadja [23]. Ennek a programnak a feladata a képpontok végső színének meghatározása. Itt történik meg a Phong-féle megvilágítási modell kiszámítása a normálvektorok és a fényforrások segítségével, valamint a képpontok saját, eredeti színe alapján. A folyamat legvégén a *fragment shader* is definiál egy kimenetet (jellemzően egy `vec4` típusú vektort), amely már közvetlenül az adott pixel tényleges, képernyőn megjelenő színét (RGBA) határozza meg [23].

Míg a 3D modellek a fenti árnyékolókat használják, a színteret körülölelő égbolt (*Skybox*) egy teljesen eltérő módszerre épül. Ennek megjelenítéséért a `skybox.vert` és `skybox.frag` fájlok felelnek. Egy égbolt esetében nem beszélhetünk hagyományos értelemben vett kétdimenziós textúrázásról; helyette a motor egy speciális, háromdimenziós köbös textúrát (*cubemap*, `samplerCube`) alkalmaz [23]. Mivel az égbolt tulajdonképpen egy hatalmas kocka belső felületét képezi, a mintavételezéshez szükséges koordinátákat maga a csúcspont térbeli iránya adja meg [23]. A *fragment shader* ezen 3D-s vektorok alapján olvassa ki a textúra pixeljeit, miközben az átadott `skyColor` változó lehetőséget a színének módosítására is.

A modellek és textúrák ebben a folyamatban válnak végleges képpé: a *vertex shader* a geometriát a megfelelő helyre mozgatja és vetíti, a *fragment shader* pedig a textúrák és a fények kombinálásával kifesti a felületeket. Ezzel a lépéssel zárul le az út, amely a matematikai adatszerkezetektől és textúrafájloktól a monitoron megjelenő, valós idejű, megvilágított háromdimenziós látványig vezet.

5.4.7. Ütközésérzékelés és fizika

Ezen a ponton már elkészültek egy grafikai szoftver alapvető funkciói. A program képes háromdimenziós, textúrázott és megvilágított objektumok megjelenítésére, a felhasználó pedig be tudja járni a környezetet. Felmerül azonban a kérdés: min sétál a játékos, mi akadályozza meg abban, hogy átmenjen a szilárd falakon, és mi történik, ha nincs alatta talaj?

A válasz az, hogy önmagában semmi. Az idáig bemutatott rendszerek kizárólag a vizuális megjelenítésért felelnek, tehát pixeleket rajzolnak a képernyőre, de semmit nem tudnak a világ fizikai tulajdonságairól. Egy fal a motor szempontjából csupán háromszögek gyűjteménye, amelyek színt és fényt kapnak, de fizikai akadályt nem jelentenek. Ha nem gondoskodunk erről külön, a játékos egyszerűen átsétál rajtuk.

Ahhoz tehát, hogy a világ ne csak megjelenjen, hanem interaktívvá váljon, szükség van egy önálló rendszerre, amely nyomon követi az entitások fizikai kiterjedését, észleli az ütközéseket, és szimulálja a gravitációt.

Első ránézésre ez magától értetődőnek tűnhet: a fizikai határnak ott kellene lennie, ahol a modell vizuálisan megjelenik. A valóságban azonban ez rendkívül összetett feladat. Egy 3D modell akár több tízezer háromszögből állhat, és ha minden egyes képkockán minden háromszög minden más háromszöggel való ütközését ellenőriznénk, az szinte felfoghatatlan számítási igényt jelent, ami pedig elfogadhatatlan teljesítményt eredményez.

A megoldás az egyszerűsítés. Az objektumokat körülhatároljuk egy egyszerűbb, láthatatlan alakzattal, ami közelíti annak alakját. A játékfejlesztésben az egyik legismertebb és legkönnyebben implementálható ilyen alakzat az úgynevezett tengelyhez igazított befoglaló téglatest (*Axis-Aligned Bounding Box*, AABB). Az AABB háromdimenziós térben egy téglatest, amely szorosan körülveszi az objektumot, és tengelyei párhuzamosak a koordinátarendszer tengelyeivel [5]. A kiszámítása egyszerű: végig kell iterálni az objektum összes csúcspontján, és mindhárom tengelyen nyomon kell követni a legkisebb és legnagyobb koordinátaértékeket [5]. A téglatest így két ponttal meghatározható: egy minimumponttal (P_{min}), amely mindhárom tengelyen a legkisebb koordinátaértékeket tartalmazza, és egy maximumponttal (P_{max}), amely a legnagyobbakat [5]:

$$P_{min} = \big(\min(x_1, \ldots, x_n),\ \min(y_1, \ldots, y_n),\ \min(z_1, \ldots, z_n)\big)$$
$$P_{max} = \big(\max(x_1, \ldots, x_n),\ \max(y_1, \ldots, y_n),\ \max(z_1, \ldots, z_n)\big)$$

ahol x_i, y_i, z_i az objektum i-edik csúcspontjának koordinátái, n pedig a csúcspontok száma.

Ahogy a neve is sejteti, az AABB nem tud forogni az objektummal együtt, ez pedig pontatlan ütközésérzékeléshez vezethet. Ez a hiányosság kiküszöbölhető az irányított befoglaló téglatest (*Oriented Bounding Box*, OBB) alkalmazásával, amely az AABB-vel ellentétben az objektummal együtt forog [5]. Bár mindkettő egy téglatest, az OBB implementációja és ütközésvizsgálata jelentősen összetettebb az AABB-nél.

Az OBB három komponensből épül fel: egy középpontból (`center`), három egymásra merőleges tengelyből (`axes`), valamint mindhárom tengelyen egy-egy félszélességből (`extent`), amely megadja, hogy a téglatest az adott irányban mekkora kiterjedéssel rendelkezik [5]. Az OBB struktúra ennek az elméletnek megfelelően került definiálásra a programkódban:

Programkód 5.2. Az OBB (Oriented Bounding Box) struktúrája

```
typedef struct {
        vec3 center;
        vec3 axes[3];
        float extents[3];
} OBB;
```

A `center` egy háromdimenziós pozícióvektor, az `axes` tömb három egységvektort tartalmaz, amelyek az OBB koordinátarendszerének tengelyeit írják le, az `extents` tömböt pedig három érték határozza meg, amelyek az OBB félszélességét adják meg a tengelyek mentén.

Egy entitás OBB-jét a `get_entity_obb` függvény számítja ki a következőképpen: először a modell alap kiterjedéseit (`base_extents`) megszorozzuk az entitás skálázási értékeivel, majd elosztjuk kettővel, így megkapjuk a félszélességeket. Ezt követően elkészítjük a forgatási mátrixot az entitás `rx`, `ry`, `rz` tulajdonságai alapján. Az OBB tengelyeit közvetlenül ebből a mátrixból olvassuk ki. A mátrix első három oszlopa adja meg az OBB lokális koordinátarendszerének irányvektorait a világtérben. Végül a középpontot úgy számítjuk ki, hogy az entitás pozíciójához hozzáadjuk a modell eltolási vektorát (`offset`), amelyet előtte ugyanezzel a forgatási mátrixszal transzformálunk. Ez biztosítja, hogy az OBB középpontja az objektum forgatásakor is a helyén maradjon [5].

Ezzel önmagában viszont még nem érünk el kézzelfogható eredményt. A tényleges fizikai interakció megvalósításához a következő lépés a karakter, tehát a kamera (játékos) OBB-jének meghatározása. Az ütközésérzékelés lényege annak ellenőrzése, hogy metszi-e egymást a kamera és egy adott entitás OBB-je. Amennyiben átfedés történik, alapesetben meg kell akadályozni a játékos továbbhaladását az akadály irányába.

A kamera OBB-jét a `get_camera_obb` függvény állítja elő, amelynek magassága dinamikusan változik attól függően, hogy a játékos áll vagy guggol. A motor minden elmozdulási szándék után meghívja a `check_camera_collision` függvényt, amely végigiterál a színtér összes aktív és szilárd (`is_solid` jelzővel ellátott) entitásán, és megvizsgálja, hogy azok metszik-e egymást. Ütközés esetén a rendszer visszavonja az elmozdulást. Ezzel sikeresen megvalósul az objektumok fizikai kiterjedésének szimulációja, meggátolva a falakon és más tárgyakon való átsétálást.

Azonban ha ütközéskor a motor minden esetben megakadályozná az elmozdulást, a játékos minden apró objektumon elakadna, például egy lépcsőfokon vagy ajtóküszöbben. Ennek megelőzése érdekében a motor egy automatikus lépcsőző funkcióval is rendelkezik. Vízszintes akadályba ütközéskor a rendszer átmenetileg megemeli a kamerát a Z tengely mentén egy előre definiált, alacsony küszöbértékkel, és megvizsgálja az átfedést ezen az új magasságon is [7]. Amennyiben itt az út már szabad, a karakter sikeresen átlép az alacsony akadályon vagy lépcsőfokon.

Az AABB-k metszésvizsgálata a legkevésbé számításigényes. Mivel ezen téglatestek tengelyei párhuzamosak a globális koordinátarendszerrel, elegendő a három főtengely (X, Y és Z) mentén külön-külön megvizsgálni a kiterjedések átfedését. Két AABB ak-

kor ütközik, ha mindhárom tengelyen egyidejűleg metszik egymást [5]. Ha az egyik doboz minimum koordinátája kisebb vagy egyenlő, mint a másik doboz maximum koordinátája, illetve a maximum koordinátája nagyobb vagy egyenlő, mint a másik minimum koordinátája, akkor az adott tengelyen van átfedés [5].

Bár az AABB vizsgálata rendkívül gyors, OBB-k esetén ez a módszer már nem alkalmazható, hiszen azok már elfordulhatnak. Az OBB-k ütközésének detektálására a játékmotor a szétválasztó tengelyek tételét (*Separating Axis Theorem*, SAT) alkalmazza.

A SAT kimondja, hogy két konvex alakzat pontosan akkor nem ütközik, ha a térben létezik legalább egy olyan tengely, amelyre mindkét alakzat merőlegesen rávetítve a keletkező vetületek nem fedik át egymást [5]. Ha találunk egy ilyen "szétválasztó tengelyt", a vizsgálat megszakítható, mert a két alakzat biztosan nem metszi egymást [5].

A játékmotor a `check_obb_collision` függvényben implementálja ezt az ellenőrzést. Háromdimenziós OBB-k esetén összesen tizenöt potenciális szétválasztó tengelyt kell megvizsgálni a biztos eredményhez: az első OBB három orientációs tengelyét, a második OBB három tengelyét, valamint ezek páronkénti vektoriális szorzatából adódó további kilenc tengelyt [5].

Az egyes tengelyek tesztelését a `test_axis` segédfüggvény végzi. Az algoritmus első lépésként a két OBB középpontját összekötő távolságvektort rávetíti az aktuális teszttengelyre egy skaláris szorzás segítségével [5]. Ezt követően mindkét OBB félszélességeit is rávetíti ugyanerre a tengelyre, majd ezek abszolút értékeit összegezve megkapja a két alakzat vetített sugarát [5]. Ha a középpontok vetített távolsága nagyobb, mint a vetített sugarak összege, a tengely szétválasztja a befoglaló téglatesteket, a függvény pedig hamis értéket visszaadva leáll. Amennyiben mind a tizenöt tengely tesztelése során átfedés található, a két OBB metszi egymást.

A tesztelések megkönnyítésére aktiválható az entitásokat körbehatároló befoglaló téglatestek kirajzolása is. A rendszer ilyenkor az entitások OBB-jeit drótvázas (`GL_LINE`) módban rajzolja ki [23].

A fizikai szimuláció elengedhetetlen része a gravitáció kezelése is. A `handle_camera_physics` függvény minden frissítési ciklusban egy folyamatos, lefelé ható gyorsulást ad hozzá a kamera függőleges sebességéhez (`vertical_velocity`). Annak meghatározására, hogy a játékos milyen objektumon áll, a rendszer sugárkövetést (*raycasting*) alkalmaz [5]. A kamera pozíciójából lefelé vetített képzeletbeli sugarakkal a `get_vertical_ray_obb_collision` függvény megkeresi a karakter alatti legmagasabban fekvő szilárd entitást és visszaadja annak azonosítóját.

Amennyiben a sugárkövetés során a karakter eléri egy objektum tetejét, a zuhanási sebesség nullázódik. A gravitáció folyamatos lefelé húzó hatása ekkor megáll, a karakter pedig földet ért (`is_grounded`) állapotba kerül.

A játékos alatt lévő entitás azonosítójának lekérdezése teszi lehetővé a mozgó, illetve forgó objektumok rendeltetésszerű kezelését is. A `handle_camera_platforms` függvény minden képkockán lekérdezi a kamera alatt lévő entitás pozíció- és rotációváltozását (delta értékeit). Míg a lineáris elmozdulásokat a rendszer egyszerűen hozzáadja a játékos koordinátáihoz, addig a forgás kezelése összetettebb. Forgó platform esetén a motor először kiszámítja a kamera relatív pozícióját a platform középpontjához viszonyítva. Ezt a vektort a platform szögelfordulásának (rotációs deltáinak) megfelelően transzformálja a `rotate_point_by_deltas` függvény segítségével, majd ebből az elforgatott vektorból állapítja meg a játékos új pozícióját. Enélkül, ha a játékos egy mozgó vagy

elforduló platformon tartózkodna, az objektum egyszerűen kicsúszna alóla.

A motor a `get_vertical_ray_obb_collision` függvény által visszaadott felületi normálvektor alapján megvizsgálja az érintett felület dőlésszögét is. Ha a felület túlságosan meredek, a karakter nem képes megállni rajta. Ilyen esetekben a motor a normálvektor irányába tolja el a kamerát, szimulálva a meredek falakról és lejtőkről való lecsúszást [5].

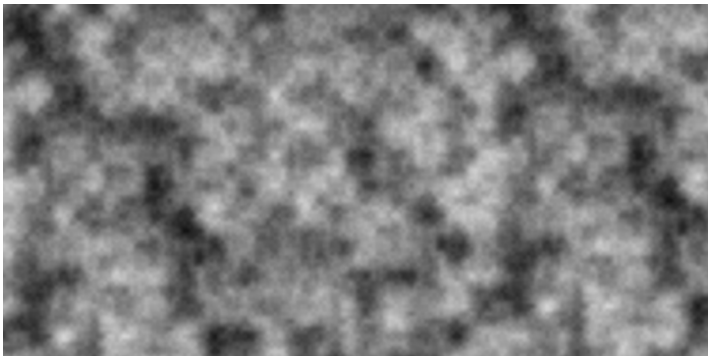
A sugárkövetés a motorban nem korlátozódik kizárólag a talajérzékelésre. Egy általános célú sugárkövető algoritmus is implementálásra került, amelyet a `raycast_scene` és a `ray_intersects_obb` függvények valósítanak meg. Ezek lehetővé teszik, hogy egy tetszőleges térbeli pontból egy megadott irányban sugarat bocsássunk ki, és megkeressük a legelső OBB-t, amelyet elmetszik. Ez használható interakciók megvalósításához, ellenőrizhető vele például az is, hogy a kamera rálát-e egy adott objektumra, de akár lövedékek esetén is használható annak megállapítására, hogy mit talált el.

Ezzel a motor alapvető fizikai és ütközéskezelési rendszere elkészült, ami lehetővé teszi a játékos szabad, mégis szabályozott mozgását a virtuális világban.

5.4.8. Domborzat

Egy játék fejlesztése során felmerül az igény, hogy látványos és változatos külső környezetet teremtsünk meg, ahol síkságok, dombok és hegyek váltakozhatnak. Bár a domborzat felépíthető lenne hagyományos 3D modellekből is, ez több szempontból sem optimális. Egyrészt kellőképpen nagy és részletes domborzat lemodellezéséhez rendkívül részletes modellekre lenne szükség, amely jelentősen megnövelné a memóriahasználatot és nagy mértékben rontaná a teljesítményt. Másrészt gondolnunk kell az ütközésdetektálásra is. A játékmotor téglatesteket (OBB) használ az objektumok körülhatárolására, így szinte képtelenség ilyen megközelítéssel kialakítani egy finom átmenetekkel rendelkező, ténylegesen bejárható domborzatot. Ehelyett a játékmotor a modern számítógépes grafikában elterjedt magasságtérkép (*heightmap*) alapú megközelítést alkalmazza.

A magasságtérkép alapvetően egy kétdimenziós, szürkeárnyalatos képfájl. A kép minden képpontja a terep egy adott pontjának magasságát reprezentálja: a fekete szín a legalacsonyabb szintet (például a tengerszintet), míg a fehér a maximális magasságot (hegycsúcsokat) jelöli [12]. A kettő közötti szürke árnyalatok határozzák meg az átmeneteket és a lejtőket.



5.1. ábra: Példa egy eljárásosan generált, szürkeárnyalatos magasságtérképre.

A magasságtérképek elkészítésére több módszer is rendelkezésre áll. Bár egyszerű képszerkesztő programokban történő manuális megrajzolás is lehetséges, ez a megközelítés leginkább a már meglévő térképek utólagos finomhangolására, vagy apróbb

módosítások elvégzésére alkalmas. Valósághű domborzatok nulláról történő felépítéséhez a modern játékfejlesztés kifejezetten erre a célra megalkotott domborzatgeneráló szoftvereket alkalmaz. Ezek lehetnek webes alapúak, de komplex, professzionális szoftverek is. Jellemzően procedurális algoritmusokra épülnek. A tervezési folyamatot nagymértékben segíti, hogy ezen programok többsége valós idejű, háromdimenziós vizuális visszacsatolást is biztosít. Így a fejlesztő nem csupán a síkbeli, szürkeárnyalatos képpontokat látja, hanem a paraméterek módosítása, vagy a terep manuális formálása közben azonnal, egy renderelt 3D-s modellen követheti nyomon a domborzat alakulását.

A procedurális domborzatgeneráló szoftverek a hagyományos véletlenszám-generálás helyett zajfüggvényeket, leggyakrabban Perlin-zajt vagy Simplex-zajt alkalmaznak [4]. Ezek az algoritmusok olyan átmeneteket hoznak létre, amelyek tökéletesen alkalmasak valósághű dombok és völgyek szimulálására. A részletgazdag domborzat elérése érdekében a generátorok jellemzően több különböző léptékű zajréteget vonnak össze: a nagyobb foltok adják az alapot, míg az kisebb, sűrűbb zajok az apró részleteket biztosítják [4]. Így a paraméterek finomhangolásával a valósághű virtuális tájat hozhatók létre.

A játékmotor a magasságtérkép betöltését és feldolgozását a `terrain.c` modul `init_terrain` függvénye végzi el. A képfájl beolvasása az `SDL_image` könyvtár segítségével történik, majd a rendszer a képet egy egységes, 24-bites RGB formátumra konvertálja. A domborzat generálása egy kétdimenziós iteráció során valósul meg, amely sorról sorra halad végig a kép magasságán és szélességén, minden képpontot egy térbeli csúcsponttá (*vertex*) alakítva. Mivel a kép szürkeárnyalatos, minden csatorna azonos értéket hordoz, így elegendő csupán az egyik, például a piros csatorna értékét kiolvasni. Az adott (`x`, `y`) koordinátájú képpont memóriacímét a rendszer úgy határozza meg, hogy a sor indexét megszorozza egyetlen sor bájtokban mért hosszával (`bytes_per_row`), majd ehhez hozzáadja az oszlopindex és a 24-bites RGB formátumból adódó 3 bájtos pixelméret szorzatát.

Mivel a 8-bites színcsatornák mindössze 256 (0 és 255 közötti) magassági szintet képesek tárolni, ez darabos, lépcsőzetes domborzatot eredményezne [12]. A motor ezért az adatokat lebegőpontos (`float`) értékké konvertálja. Ezt az értéket az algoritmus normalizálja egy 0.0 és 1.0 közötti tartományra, majd megszorozza a globális vertikális skálázási tényezővel (`scale_z`). Így a domborzat tetszőleges méretarányhoz igazítható:

Programkód 5.3. Magasságérték kinyerése és skálázása a pixeladatokból

```
int bytes_per_row = surface->pitch;
Uint8 r = pixels[y * bytes_per_row + x * 3];
float normalized_height = (float)r / 255.0f;
float height = normalized_height * scale_z;
terrain->heights[x][y] = height;
```

A csúcspontok vízszintes (X és Y) elhelyezkedését a rácsban elfoglalt indexük és egy horizontális skálázó érték (`scale_xy`) szorzata adja meg. A valósághű megvilágítás eléréséhez elengedhetetlen a felületi normálvektorok meghatározása is. Ezt a motor csúcspontonként a szomszédos magasságértékekből számított érintővektorok segítségével végzi el [10]. Az algoritmus a grafikai programozásban elterjedt centrális differencia (*central difference*) módszerét alkalmazva megvizsgálja a közvetlenül szomszédos rácspontok magasságkülönbségét a vízszintes és függőleges tengelyek mentén [12]. Ezekből a differenciákból, a tangens vektorok vektoriális szorzatának leegyszerűsített, optimalizált számításával a rendszer egy normalizált háromdimenziós vektort képez [10].

A csúcspontok és a felületi normálisok meghatározása után a motornak fel kell építenie magát a megjeleníthető geometriát. A grafikus gyorsítók (GPU-k) alapvetően háromszögekből (primitívekből) építik fel a 3D-s objektumokat, így a rácspontok halmazát egy összefüggő háromszöghálóvá (*mesh*) kell alakítani [19]. Ezt az indexelés (*index buffer*) technikájával valósítja meg a rendszer. Mivel egy $W \times H$ méretű rács összesen $(W-1) \times (H-1)$ darab négyszögletes cellát (*quad*-ot) alkot, az algoritmus egy dupla ciklus segítségével ezen cellák mentén halad végig, és minden cellát két háromszögre bont [12].

Az adatok hatékony tárolásához a motor az EBO (*Element Buffer Object*) technikát alkalmazza. Mivel egy összefüggő domborzati rács belső pontjain akár hat háromszög is osztozhat, az EBO használata nélkül minden egyes közös csúcspont adatait többszörösen kellene eltárolni a memóriában. Az indexelt rajzolás miatt azonban minden csúcspontot (*vertex*) csak egyszer tárolunk el a VBO (*Vertex Buffer Object*) bufferben, az EBO pedig csupán kisméretű indexeket tartalmaz, amelyek megadják a háromszögek összeállítási módját [19]. Ez a megoldás jelentős mennyiségű videómemóriát takarít meg és növeli a teljesítményt.

A textúrázás során a motor ahelyett, hogy rögzített számú ismétlést feszítene a teljes terepre, a rendszer a világkoordináták és egy méretarány (`meters_per_tile`) hányadosaként számítja ki az UV koordinátákat. Ez biztosítja, hogy a textúra fizikai mérete és részletgazdagsága konzisztens maradjon a magasságtérkép kiterjedésétől függetlenül, megakadályozva a textúra nagyobb eltorzulásait.

Miután a vertexek, a normálvektorok, a textúrakoordináták, valamint a hozzájuk tartozó indexek kiszámításra kerültek, az adatok feltöltésre kerülnek a videókártya memóriájába. Ezt követően a motor a rendszermemóriából felszabadítja az adatokat. Az egyetlen adat, amely megőrzésre marad továbbra is megőriz, az a `heights` kétdimenziós mátrix, amely a fizikai interakciók és az ütközésvizsgálat alapjául szolgál.

Ahhoz, hogy a karakter sétálni tudjon a domborzaton, a motornak minden képkockán meg kell határoznia a talaj pontos magasságát a kamera aktuális pozíciója alatt. Mivel a játékos pozíciója folyamatos lebegőpontos érték, szinte sosem esik egybe a terep konkrét rácspontjaival. Ennek kiküszöbölésére a `get_terrain_height` függvény bilineáris interpolációt alkalmaz [12]. Az algoritmus meghatározza azt a négy rácspontot, amely a kamerát aktuálisan közrefogja, majd a játékos cellán belüli elhelyezkedése alapján súlyozott átlagot von a négy magasságértékből [12]. Ez a matematikai eljárás biztosítja, hogy a karakter rángatózás nélkül, sima pályán haladhasson végig a lejtőkön.

A fizikai rendszer kiegészül egy meredekségvizsgálattal is. A `handle_camera_movement` függvény a tervezett lépés előtti és utáni magasságkülönbségből kiszámítja a lejtő meredekségét. Amennyiben ez meghalad egy előre definiált küszöbértéket, a motor visszavonja a lépést. Ez megakadályozza, hogy a játékos irreálisan meredek hegyoldalakon másszon fel.

5.4.9. Részecskerendszer

A vizuális élmény fokozása érdekében a `particle.c` modulban egy részecskerendszer került implementálásra, amely lehetővé teszi olyan dinamikus effektek megjelenítését, mint például a tűz vagy a füst. Ez a terület alapjaiban más megközelítést igényel, mint a statikus 3D objektumok kirajzolása.

A tűz és a hasonló jelenségek nem egyetlen, összefüggő modellből állnak, hanem több száz vagy akár több ezer apró, rövid életű elemből, amelyek pozíciója és színe

folyamatosan változik. A hagyományos renderelési eljárás során minden egyes objektum kirajzolása egy különálló rajzolási parancsot (*draw call*) igényel [7]. Míg egy átlagos virtuális tér csupán néhány tíz objektumból épül fel, addig egyetlen részecskeeffektus önmagában több százszor annyi hívást generálna, mint a színtér többi része összesen. Ez a kommunikációs teher leamortizálná a teljesítményt, így egy optimális megoldásra volt szükség.

A megoldást a (*Hardware Instancing*) és a *billboarding* technika együttes alkalmazása jelenti [23]. Az instancing lehetővé teszi, hogy a CPU egyetlen utasításban (`glDrawArraysInstanced`) küldje el az összes aktív részecske egyedi adatait a videókártyának [23]. Az adatok hatékony átvitelét a `glVertexAttribDivisor` hívások segítik, amelyek biztosítják, hogy a pozíció, a szín és a méret attribútumok ne vertexenként, hanem példányonként frissüljenek a GPU memóriájában [23].

A részecskék valójában kétdimenziós négyzetek (*quad*ok), amelyeket háromszögekből építünk fel, mivel a videókártya háromszögeket képes megjeleníteni [23]. Önmagában a kétdimenziós alakzatok használata értelmetlen lenne, hiszen csak közvetlenül szemből vagy hátulról látszódnának, oldalról nézve pedig csak papírvékony vonalakat vagy egyáltalán semmit sem látnánk. Ezért elengedhetetlen a *billboarding* technika alkalmazása, aminek a lényege, hogy a részecskéket minden képkockában a kamera felé forgatja, így fenntartva a térbeliség illúzióját.

Ennek megvalósítása a `particle.vert` shaderben történik. A motor a kamera jobb (`cameraRight`) és felső (`cameraUp`) egységvektorai alapján minden egyes képkockán úgy tolja el a részecskék vertexeit, hogy azok síkja merőleges maradjon a nézői irányra. A számítás során felhasznált bemeneti változók közül az `aQuadVert` a részecskét alkotó négyzet lokális eltolását jelöli a középponthoz képest, az `aPos` az effekt példányonkénti középpontját adja meg a 3D-s térben, az `aSize` a méretezésért felel, a `cameraRight` és `cameraUp` egységvektorok pedig a kamera síkjával párhuzamos vízszintes és függőleges irányokat határozzák meg.

A színtérben elfoglalt végleges pozíció (`worldPos`) kiszámítása a klasszikus *billboarding* matematikai modellje szerint történik [23]:

$$\begin{aligned}\texttt{worldPos} = \texttt{aPos} + (\texttt{cameraRight} \cdot \texttt{aQuadVert.x} \cdot \texttt{aSize}) \\ + (\texttt{cameraUp} \cdot \texttt{aQuadVert.y} \cdot \texttt{aSize})\end{aligned}$$

Egy részecske legfontosabb tulajdonságai a pozíció, a sebesség és a szín, valamint az élettartam. A részecskék kibocsátását (*emission*) az `emit_particle` függvény végzi, itt kapja meg a részecske a kezdeti tulajdonságait. A motor egy fix méretű tömböt hoz létre a memóriában, amelyben kezdetben minden részecske inaktív. Új részecske létrehozásakor a rendszer az `emit_particle` hívásával megkeresi az első inaktív elemet, és újrahasznosítja azt az új paraméterek beállításával. A függvénynek megadható, hogy milyen pozíción, milyen sebességgel és milyen élettartammal hozza létre az adott részecskét. Realisztikus megjelenéshez mindezeket véletlenszerűen kell előállítani bizonyos határok között.

A részecskék állapotának frissítése az `update_particles` függvényben történik. A motor minden képkockában csökkenti az aktív részecskék élettartamát, és a sebességük alapján elmozdítja a pozíciójukat. Ugyanitt történik az átlátszóság dinamikus számítása is: ahogy a részecske élettartama a végéhez közeledik, a szín alfa értéke fokozatosan csökken, így az effektek fokozatosan elhalványulnak.

A megfelelő vizuális hatást a `particle.frag` *fragment shader* és a renderelési beállítások érik el. A shader a `discard` utasítással eldobja azokat a pixeleket, amelyek túlságosan átlátszóak vagy sötétek, ami számítási időt takarít meg [23]. A renderelés előtt beállított additív színkeverés (`glBlendFunc(GL_SRC_ALPHA, GL_ONE)`) pedig azt eredményezi, hogy az egymást fedő részecskék fényereje összeadódik [23]. Ennek köszönhetően a lángok izzó hatást kapnak.

5.4.10. Felhasználói felület (UI)

A korábban bemutatott ortogonális vetítési módot alkalmazva az `ui.c` modul felel a kétdimenziós tartalmak, mint például a menük és feliratok képernyőre rajzolásáért. A modul három elemtípus megjelenítését támogatja: téglalapokat (`draw_rectangle`), képeket (`draw_image`) és szövegeket (`draw_text`).

Attól függetlenül, hogy éppen melyik elemtípust szeretnénk kirajzolni, a megjelenítés alapja minden esetben ugyanaz az egyetlen, két háromszögből álló egységnégyzet (*quad*), amelyet a motor az inicializálás során egy VBO-ba tölt fel. A különböző méretű UI-elemek létrehozása azt jelenti, hogy ezt az alapnégyzetet skálázzuk tetszőleges szélességűvé és magasságúvá [23]. Mivel a felületnek a térbeli elrendezéstől függetlenül mindig a 3D-s színtér előtt kell megjelennie, a motor a rajzolás idejére kikapcsolja a mélységvizsgálatot (`glDisable(GL_DEPTH_TEST)`), az átlátszóság helyes kezeléséhez pedig bekapcsolja az alfa-keverést [23].

A négyzet eltolását és skálázását a `ui.vert` *vertex shader* végzi. A shader bemenetként egy négykomponensű vektort (`vertex`) kap, amelynek első két eleme (`xy`) a pozíciót, a második két eleme (`zw`) pedig a textúrakoordinátákat tartalmazza [23]. A csúcspontok végleges pozícióját a képernyőn egy három lépésből álló mátrix-vektor szorzás határozza meg. Első lépésként 2D-s pozíciót a shader egy 3D-s térbeli ponttá egészíti ki. Ezt megszorozza az elemek skálázásáért és eltolásáért felelős modellmátrixszal (`model`), amely az egységnégyzet csúcspontjait a kívánt pixelméretre és pozícióra transzformálja. Végül az eredményt megszorozza az ortogonális projekciós mátrixszal [23].

A `ui.frag` *fragment shader* rugalmas kialakításának köszönhetően képes az egyszerű téglalapokat és a textúrázott képeket is kezelni. Ezt a `useTexture` logikai (*boolean*) `uniform` változó határozza meg. Ha az értéke hamis, a shader kizárólag a megadott színnel tölti ki a téglalapot. Ellenkező esetben ráfeszíti az átadott textúrát, amelynek színeit a shader megszorozza a kapott `color` vektorral is [23]. Ez lehetővé teszi, hogy az alapértelmezetten fehér színű képek átszínezhetők legyenek.

A felhasználói felület legkomplexebb része a szöveges információk megjelenítése, mivel a GPU hardveresen nem képes betűtípusokat értelmezni és rajzolni [23]. Ennek áthidalására a szövegmegjelenítés valójában egy képalkotási folyamat. A legelső lépés a megfelelő TrueType betűtípus betöltése. A motor a `get_ui_font` függvény segítségével egy külön gyorsítótárban tárolja a már megnyitott betűtípusokat, így azokat csak egyszer kell a memóriába tölteni. Ezt követően a motor az `SDL_ttf` könyvtárat használja, amely "rárajzolja" a betűket egy átlátszó, ideiglenes vászonra (`SDL_Surface`) [21]. Ez az elkészült vászon gyakorlatilag egy 2D-s kép, amelyet a motornak ezután át kell konvertálnia, majd textúraként fel kell töltenie a videókártya memóriájába. Innentől kezdve a szöveg textúraként viselkedik, amelyet a motor ugyanúgy kezel, mint bármilyen más képet [23].

Mivel ez a folyamat meglehetősen lassú, minden egyes képkockán történő végrehaj-

tása teljesítményromlást okozna. Ennek elkerülésére a motor egy szöveggyorsítótárat is implementál. A `draw_text` függvény a szöveg első megjelenítésekor eltárolja a legenerált textúra azonosítóját, szélességét és magasságát. A következő képkockákon a rendszer ellenőrzi, hogy a kért szöveg, a betűtípus és a méret szerepel-e már a gyorsítótárban. Amennyiben igen, a motor azonnal a már VRAM-ban lévő textúrát használja fel.

A rugalmas elrendezés érdekében a szöveget kirajzoló függvény támogatja a szöveg igazítását is (balra, középre vagy jobbra), amelyet a motor a legenerált textúra szélességének ismeretében az X koordináta automatikus eltolásával valósít meg.

5.4.11. Hangrendszer

Az alkalmazás hangkezeléséért a `sound.c` modul felel, amely az `SDL_mixer` könyvtárra épülve biztosítja a hangeffektek és háttérzenék lejátszását. A motor inicializálásakor az `init_sound` függvény beállítja az audiokontextust, és egymástól független csatornákat foglal le, amelyeken a hangok párhuzamosan szólalhatnak meg [7].

A hangrendszer kétféle lejátszási módot támogat: a hagyományos kétdimenziós hangokat és a térbeli hangokat. A kétdimenziós hangok (`play_sound`) esetében a hangerő konstans, és a hangforrás független a játékos helyzetétől, ami ideális a felhasználói felület visszajelzéseihez vagy a globális háttérzenékhez.

A térbeli hangok (`play_sound_3d`) kezeléséhez a motor egy nyilvántartja az aktív csatornák térbeli pozícióját (X, Y, Z), alapértelmezett hangerőszintjét és maximális hallhatósági távolságát (`max_distance`). Annak érdekében, hogy a hangok hangereje reagáljon a játékos mozgására, a fő ciklus minden képkockában meghívja az `update_sounds` függvényt, átadva neki a kamera aktuális paramétereit.

A térbeli hatás frissítése az `update_channel_3d` függvényben történik, amely egy klasszikus lineáris csillapítási modellt alkalmaz a távolság függvényében [7]. A motor az euklideszi távolságképlet segítségével kiszámítja a kamera és a hangforrás közötti távolságot. Ezt viszonyítva a hanghoz beállított maximális hatótávhoz (`max_distance`), a rendszer egy lineáris normalizációt hajt végre: ahogy a játékos távolodik a forrástól, az `SDL_mixer` csatornájának hangereje (`Mix_Volume`) arányosan csökken [7].

5.5. A szkriptelési rendszer implementálása

5.5.1. A C és a Lua összekapcsolása

A játékmotor rugalmasságát a Lua szkriptnyelv integrálása adja, amely lehetővé teszi, hogy a motor forráskódjának újrafordítása nélkül vezérelhető és módosítható legyen a játékmenet vagy a felhasználói felület. A két réteg kommunikációja a `scripting.c` modulban valósul meg a hivatalos Lua C API-t alkalmazva.

A rendszer inicializálása az `init_scripting` függvényben történik három fő lépésben:

1. **A Lua állapot (*Lua state*) létrehozása:** A `luaL_newstate` függvény hívásával a motor inicializál egy független, elszigetelt környezetet. Ez az állapot maga a Lua virtuális gép. Tartalmazza a globális változókat, a betöltött szkripteket, valamint azt a vermet (*stack*), amely a C és a Lua közötti kommunikációt teszi lehetővé [8].

2. **A standard könyvtárak betöltése:** A `luaL_openlibs` függvény meghívásával a motor betölti a Lua alapkönyvtárait (például a `math`, `string` és `table` modulokat) [8].

3. **A C függvények regisztrálása:** Ahhoz, hogy a Lua szkriptek kezelni tudják a motor logikáját, láthatóvá kell tenni számukra a C-ben megírt funkciókat. A motor ezt a `lua_register` függvény segítségével éri el. Ennek során egy C függvény hozzárendelődik egy Lua környezetben használható névhez (például `createObject`), így hidat képezve a két réteg között [8].

A Lua környezet és a C motor közötti kapcsolat alapvetően kétirányú:

1. C függvények hívása Lua-ból: A játékmotor belső függvényeit a Lua nem képes közvetlenül meghívni, ezért úgynevezett burkoló (*wrapper*) függvények kerülnek alkalmazásra, amelyek áthidalják a két nyelv közötti különbségeket. Minden ilyen burkolófüggvény kizárólag a `lua_State` mutatót várja bemenetként [8]. A függvények első feladata, hogy sorban leemeljék a virtuális veremről a paramétereket a várt típusuknak megfelelő makrókkal (például a `luaL_checkinteger` vagy a `luaL_checkstring` segítségével) [8]. Ezután az átvett paraméterekkel meghívják a motorban ténylegesen definiált C függvényeket. Végül felhelyezik a veremre a Lua-nak szánt visszatérési adatokat a `lua_pushnumber` és hasonló makrókkal. Visszatérési értékük minden esetben a veremre újonnan felhelyezett változók száma [8]. Minden esetben rendelkeznek valamilyen visszatérési értékkel, sikertelen végrehajtás esetén például hamis értéket adnak vissza a `lua_pushboolean` használatával.

2. A játékmotor eseményeinek küldése a Lua felé: A szkriptelés alapja az eseményekre való reagálás. A játék futása során fellépő eseményeket a motor közvetíti a szkriptek felé, amelyek erre reagálhatnak. Az események bekövetkezésének jelzése mellett a rendszer paramétereket is átadhat (például egy egérkattintás X és Y koordinátáit). A kihívást az jelenti, hogy a különböző események eltérő számú és típusú paraméterekkel rendelkeznek. Mivel a C egy szigorúan típusos nyelv, alapesetben minden egyedi eseménytípus továbbításához külön függvényt kellene implementálni, ami nem lenne kedvező döntés.

Ezt a problémát a motor a C `<stdarg.h>` könyvtárának alkalmazásával oldja meg, amely lehetővé teszi egyetlen, változó paraméterlistájú függvény, a `trigger_lua_event` implementálását. A függvény kötelező paraméterei az esemény neve és a további argumentumok formátuma, ezt követően pedig a három pont (`...`) jelzi a tetszőleges számú és típusú paramétert. Például egy lebegőpontos szám (*float*) és egy string átadásához az `"fs"` formátumot kell megadni második paraméterként. A dinamikus paraméterátadás a következő makrók segítségével történik a C kódban:

- `va_list`: Létrehoz egy mutatót, amely a memóriában a változó argumentumok listájára hivatkozik.

- `va_start`: Inicializálja a mutatót. A mutató kezdőértéke az első változó argumentum memóriacíme lesz.

- `va_arg`: Kiolvassa a memóriából a következő argumentumot. A motor a `format` sztring karaktereit (például `'i'` *integer*, `'f'` *float*, `'s'` *string*, `'b'` *boolean*) használja fel arra, hogy átadja a `va_arg` makrónak, hogy milyen típusként értelmezze a következő adatot. A kiolvasott értékeket a motor a megfelelő API hívásokkal azonnal fel is helyezi a veremre.

- `va_end`: Lezárja a folyamatot.

Ezzel sikeresen felépült a kétirányú híd a C és a Lua között. A `scripting.c` utolsó lépésként lefuttatja a gyökérkönyvtárban található `lua_core/init.lua` fájlt, amely átadja a vezérlést a szkriptelési keretrendszernek. A keretrendszer tartalmazza a magas szintű játékelemek definícióit, az esemény- és erőforrás-kezelőt.

5.5.2. Eseménykezelés

Míg a játékmotor logikája egy végtelen ciklusban (*game loop*) fut, addig a Lua szkriptek eseményvezérelt elvre épülnek: nem folyamatosan futnak, hanem specifikus eseményekre iratkoznak fel, és azokra reagálnak [16]. Ennek a működését a `lua_core/event.lua` modul biztosítja.

A fejlesztők az `addEventHandler` függvénnyel hozhatnak létre új eseménykezelőket. A hívás során meg kell adni a figyelni kívánt esemény nevét, illetve a végrehajtandó saját, szkriptben definiált függvényt. Opcionálisan megadható, hogy az eseménykezelő melyik konkrét játékelemhez kapcsolódjon. Ha ez a paraméter nincs megadva, az alapértelmezett érték a `"root"` lesz, ami azt jelenti, hogy a függvény az esemény forrásától függetlenül mindig lefut. Ha azonban megadjuk egy játékelem azonosítóját, a függvény csak akkor hajtódik végre, ha az esemény kifejezetten azzal az objektummal történt meg.

Az eseménykezelők egy globális táblában (`eventHandlers`) kerülnek eltárolásra, ahol a kulcs az esemény neve, az értékek pedig a hozzájuk kapcsolt függvények listái.

Az eseménykezelők tényleges lefuttatása a `triggerEvent` függvénnyel történik, amelyet meghívhat egy szkript vagy maga a játékmotor is. A hívás során átadásra kerül az esemény neve, a forrás entitás (`sourceElement`), valamint a további dinamikus paraméterek. Amikor ez a függvény lefut, a rendszer lekérdezi az `eventHandlers` táblából az adott eseményhez tartozó függvényeket, majd végigiterál rajtuk és lefuttatja őket [16]. A végrehajtás előtt egy `source` nevű, globálisan elérhető változóba menti az eseményt kiváltó entitást. Ez jelentősen kényelmesebbé teszi a szkriptelést, mivel az eseménykezelő függvények a beépített `source` változón keresztül könnyedén hivatkozhatnak a forrásra anélkül, hogy azt külön paraméterként kellene átvenniük.

A feleslegessé vált eseménykezelők a `removeEventHandler` függvény segítségével bármikor dinamikusan törölhetők.

5.5.3. Adatkezelés

A játékmenet fejlesztése során gyakran felmerül az igény arra, hogy a játékelemeket könnyedén elláthassuk dinamikusan változtatható adatokkal (például egy mozgó objektum célkoordinátájával). Erre a célra szolgál a `lua_core/data.lua` modul.

A rendszer egy táblában tárolja az úgynevezett entitásadatokat (*element data*), az eseménykezelőkhöz hasonló struktúrában. A táblában a kulcsok az objektumok azonosítói, az értékek pedig a hozzárendelt adatok neveit és azok értékeit tartalmazó beágyazott táblák. A fejlesztők a `setElementData` és `getElementData` függvényeken keresztül manipulálhatják ezeket az adatokat.

Különös figyelmet kellett fordítani arra, hogy ha a játékmotor töröl egy objektumot, akkor a hozzá tartozó adatok is törlődjenek a szkriptelési keretrendszerben. Ehhez egy `destroyElement` nevű burkolófüggvény került alkalmazásra az `elements.lua`

modulban. Amikor egy szkript törölni kíván egy entitást, a szkript már nem közvetlenül a játékmotor natív függvényét hívja meg. A rendszer első lépésként meghívja a `clearElementData` függvényt, amely törli az entitáshoz tartozó összes adatot. Ezt követően a keretrendszer továbbítja a hívást, így a motor végrehajthatja az entitás megsemmisítését.

5.5.4. Időzítők

A játékmenet szkriptelése során elengedhetetlen bizonyos függvények késleltetett vagy meghatározott időközönként ismétlődő végrehajtása. A `lua_core/timer.lua` modul egy képkockasebességtől független időzítőrendszert valósít meg.

Az új időzítők létrehozására a `setTimer` függvény szolgál, amely négy alapvető paramétert vár: a végrehajtandó függvényt, az időköz hosszát milliszekundumban, a végrehajtások számát, valamint a végrehajtandó függvény opcionális paramétereit. Ha a végrehajtások száma nullára van beállítva, azt a rendszer végtelenített futásként értelmezi, amelyet a `math.huge` konstanssal reprezentál. Az aktív időzítők az `activeTimers` nevű táblában kerülnek eltárolásra.

Az időzítők működése az eseménykezelő rendszeren alapszik. A modul `processTimers` nevű függvénye feliratkozik az `onRender` eseményre, ami azt jelenti, hogy a játékmotor minden egyes képkocka kirajzolásakor meghívja azt.

A függvény bemenetként megkapja az előző képkocka óta eltelt időt (*delta time*), amelyet hozzáad az aktív időzítők `elapsed` (eltelt) számlálójához. Amikor ez az idő eléri a megadott értéket, lefut a megadott függvény. Ezzel a megközelítéssel az időzítés teljesen független attól, hogy a játék milyen képkockasebességgel fut [7].

A végrehajtás után a rendszer növeli a lefutások számát, és ha egy időzítő elérte a kívánt ismétlésszámot, automatikusan törlődik. Rendelkezésre áll a `destroyTimer` függvény is, amellyel bármely aktív időzítő megszakítható és törölhető.

5.5.5. Zónák

A `lua_core/zone.lua` modul lehetővé teszi térbeli területek (zónák) létrehozását, valamint annak érzékelését, amikor a játékos belép egy ilyen zónába, vagy elhagyja azt. A rendszer két típust támogat: a gömb (*sphere*) és a téglatest (*cuboid*) alakú zónákat.

Új zóna a `createZoneSphere` (középpont és sugár megadásával), valamint a `createZoneCuboid` (bal alsó sarok pozíciójának, illetve a szélesség, mélység és magasság megadásával) függvényekkel hozható létre, adataik pedig a `zones` táblában tárolódnak. A tárolt adatok között szerepel egy `isPlayerInside` nevű logikai változó is, amely nyomon követi, hogy a játékos az adott zónán belül tartózkodik-e.

Ha a rendszer csak a pozíciókat vizsgálná, a zónába lépés eseményét minden egyes képkockán detektálná. Ennek elkerülésére szolgál az `isPlayerInside` változó. Ha a játékos belép a zónába, a változó értéke igazra vált, és a rendszer meghívja az `onZoneHit` eseményt. Mivel az `isPlayerInside` értéke már igaz, a következő képkockákon nem fut le újra az esemény. Hasonló logika mentén működik a kilépés detektálása az `onZoneLeave` eseménnyel.

A zónák ellenőrzése tehát folyamatosan, az `onRender` eseményhez kötve történik. A rendszer minden képkocka kirajzolásakor lekérdezi a kamera pozícióját, majd egy segédfüggvény segítségével ellenőrzi az ütközést a zóna típusa alapján.

Gömb alakú zónák esetén a motor a háromdimenziós euklideszi távolságképletet alkalmazza a kamera és a gömb középpontja között. Ha a kiszámított távolság kisebb vagy egyenlő, mint a zóna sugara, a játékos a zónán belül tartózkodik [5]:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Téglatest alakú zónákhoz a rendszer egy klasszikus AABB ütközésdetektálást végez el.

5.5.6. Felvehető tárgyak

A felvehető tárgyak (*pickups*) olyan kisméretű, a színtérben megjelenő interaktív elemek (például gyűjthető érmék), amelyek nem rendelkeznek fizikai kiterjedéssel, így a játékos áthaladhat rajtuk. Céljuk, hogy ha a játékos interakcióba lép velük (belesétál a területükbe), eltűnjenek, és a játékos valamilyen jutalomban részesüljön. Ezt a mechanikát a `lua_core/pickup.lua` modul implementálja, amely a motorban a tárgyakat normál objektumokként hozza létre, megfelelő állapotjelzőkkel ellátva.

Új felvehető tárgy a `createPickup` függvénnyel hozható létre. A keretrendszer első lépésként generál egy objektumot a megadott koordinátákon, majd a `setElementCollisionsEnabled` hívással letiltja annak fizikai ütközését. A detektálás érdekében a rendszer minden pickup köré egy gömb alakú zónát helyez el. A 3D-s objektumok és a hozzájuk tartozó zónák a `zoneToPickup` táblában vannak összekapcsolva.

A vizuális megjelenést, a folyamatos forgó és lebegő animációt a rendszer az `onRender` eseményre feliratkozva végzi el. A forgás egy egyszerű tengely körüli rotáció, a lebegést pedig egy időalapú szinuszfüggvénnyel számítja ki a motor [7].

Amikor a játékos átsétál a tárgyat körülvevő zónán, a rendszer a zónarendszeren keresztül detektálja az ütközést. A `zoneToPickup` tábla alapján azonosítja a hozzá tartozó objektumot, és meghívja az `onPickupHit` eseményt.

5.5.7. Részecskerendszer

A játékmotorban implementált részecskerendszer önmagában nem ismeri a "tűz" vagy "füst" fogalmát, csupán részecskék megjelenítésére alkalmas a megadott paraméterek alapján. A részecskerendszer magasabb szintű vezérlését a `lua_core/particle.lua` modul végzi.

A jelenlegi implementáció tűzeffektusok létrehozását (`createFire`) biztosítja. A függvény meghívásakor meg kell adni az effektus pozícióját, a részecskék sűrűségét, valamint a tűz kiterjedését és méretét. A létrehozott tűzeffektusokat a rendszer az `activeFires` táblában tárolja.

A részecskék viselkedését egy `onRender` eseményhez kapcsolt logika határozza meg. A rendszer minden egyes képkocka kirajzolásakor végigiterál az aktív tüzeken, és a beállított értékeknek megfelelő számú új részecskét generál.

A természetes vizuális hatás elérése érdekében a szkript a `math.random()` függvény segítségével minden egyes részecskénél apró, véletlenszerű eltéréseket alkalmaz a pozíciójukban, sebességükben és élettartamukban egyaránt. Így a részecskék nem egyetlen pontból szállnak fel, hanem a beállított területen belül véletlenszerűen szétszórva. Felfelé egy erősebb, véletlenszerű sebességet kapnak a részecskék, oldalirányba pedig szintén véletlenszerű, de jóval kisebb értékeket. A részecskék kezdőmérete és élettartama szintén véletlenszerű.

A kiszámolt paramétereket a szkript végül átadja az `emitParticle` függvénynek, ezáltal a motor megjeleníti a részecskéket a meghatározott formában. Ez a megközelítés rendkívül rugalmas, ugyanis a paraméterek módosításával viszonylag könnyen létrehozhatók teljesen új vizuális effektek is.

5.5.8. Erőforrások és az erőforrás-kezelő

Az előző alfejezetekben bemutatott Lua keretrendszer egy alapvető eszköztárat alkot. A tényleges játékmenet vagy a felhasználói felület fejlesztése során a programozók tipikusan már nem ezeket a fájlokat módosítják. Ehelyett a keretrendszer API-jára támaszkodva úgynevezett erőforrásokat (*resource*) hoznak létre, amelyek már teljes mértékben a saját munkájukat képezik, és a játék specifikus funkcióit valósítják meg.

Egy erőforrás logikailag és fizikailag is egy elkülönülő, önálló mappában elhelyezkedő csomag. Ezek a csomagok teljesen modulárisak: külön-külön indíthatók, leállíthatók vagy újraindíthatók, ugyanakkor képesek egymás kiegészítésére is. Egy erőforrás nemcsak Lua szkriptfájlokat tartalmazhat, hanem magában foglalhatja a működéshez szükséges egyéb fájlokat (*asset*eket), például modelleket, textúrákat, képeket és hangeffekteket. Ezáltal a *resource* valóban egy önálló, hordozható egység.

Az erőforrások két típusra lettek bontva. A `"gamemode"` típusú erőforrások egy játékmód gerincét adják. A keretrendszer biztosítja, hogy egyszerre csak egy ilyen típus futhat: ha a rendszer egy új játékmódot indít el, az előzőleg futó játékmódot automatikusan leállítja. A többi, `"script"` típusú erőforrás pedig ezen játékmódok kiegészítésére, specifikus részfeladatok ellátására szolgál.

A Lua nyelvben a játékmotor függvényei (API) és a saját alapvető függvényei (például matematikai függvények) egy központi, globális névtérben, a `_G` táblában helyezkednek el [8]. Ha minden szkript közvetlenül ebben a térben futna, az erőforrások könnyedén felülírnák egymás változóit és függvényeit, ami instabilitáshoz vezetne [8]. Ennek elkerüléséért és a memóriakezelésért a `lua_core/manager.lua` modul felel. Amikor a manager betölti a szkripteket, azokat egy külön, egyedi futtatási környezetben (`resourceEnvironment`) indítja el. Ez a környezet a Lua metatábla (*metatable*) funkcióját kihasználva egy visszautalást (`__index = _G`) tartalmaz a globális térre [8]. Ennek köszönhetően, amikor egy szkript meghív egy függvényt, a rendszer először a lokális névterében (`resourceEnvironment`) keresi azt [8]. Ha ott nem találja, akkor a globális `_G` táblából olvassa ki az értéket [8].

Az erőforrások lokális környezetében a manager szándékosan felülírja az alapvető, játékelemek létrehozására szolgáló globális függvényeket (például `createObject`, `setTimer`, `addEventHandler`) a saját burkoló függvényeivel. Ennek a működésnek több előnye is van. Egyrészt a burkolófüggvények a `resolvePath` belső metódus segítségével döntenek a hivatkozott paraméterek (fájlútvonalak) helyéről: ha a kapott útvonal normál fájlnév, a rendszer azt az adott erőforrás saját állományának tekinti és kiegészíti annak abszolút útvonalával. Ha viszont perjellel kezdődik, a manager globális hivatkozásként kezeli, és az állományt a játékmotor gyökérkönyvtárában keresi. Továbbá a létrehozott entitások az `activeResources` táblán belül az őket létrehozó erőforrás neve alá kerülnek, típusonként (például objektum, hang) kategorizálva. Ez azt jelenti, hogy a *resource manager* tisztában van azzal, hogy melyik entitás melyik erőforrásban jött létre.

Ennek a legnagyobb előnye az erőforrások leállításakor mutatkozik meg igazán. A leállítás során a manager végigiterál az adott *resource* által létrehozott összes entitás-

típuson, és azokat automatikusan megsemmisíti. Így az erőforrások felszabadításával nem a fejlesztőknek kell foglalkoznia, és garantáltan nem marad hátra semmilyen memóriaszemét az egyik erőforrásból a másikra váltáskor.

Az erőforrások tartalmát és egyéb paramétereit a `meta.lua` fájlok fogják össze. Ez a fájl minden csomag kötelező belépési pontja. Ebben definiálhatók olyan alapadatok, mint a *resource* neve, leírása, készítője, típusa, verziószáma, de itt kell megadni a betöltendő szkriptfájlokat is. A `meta.lua` fájl például a következőképpen nézhet ki:

Programkód 5.4. Példa egy meta.lua fájlra

```
name "My parkour level"
author "John"
description "Reach the goal before the time runs out!"
version "1.0"
type "gamemode"
scripts {
        "scripts/main.lua",
        "scripts/utils.lua",
}
```

A `meta.lua` fájl feldolgozását szintén egy külön környezetben, a `metaEnvironment` végzi el. Ez a környezet szándékosan nem lát rá a globális névtérre és a játékmotor API-jára. Kizárólag a konfiguráció beolvasásához szükséges funkciókat ismeri. A `meta.lua` a `loadfile` függvénnyel kerül betöltésre és a `pcall` (*protected call*) hívással fut le [8].

A `meta` fájl nem egy hagyományos konfigurációs fájl, hanem egy Lua szkript, ahol minden sor függvényhívás. A Lua nyelv egyik sajátossága, hogy egyetlen argumentum (string vagy tábla) esetén el lehet hagyni a függvényhívás zárójeleit, emiatt formázható úgy, hogy egy tényleges, könnyen olvasható és módosítható konfig fájlnak tűnjön [8].

5.5.9. Pályabetöltés

A játéklogika és a szkriptek önmagukban még nem alkotnak teljes játékélményt, ehhez a virtuális környezet felépítése is szükséges. Bár technikailag lehetséges az objektumokat, fényeket, effekteket és hangokat közvetlenül a fő szkriptfájlokban létrehozni, ez a megközelítés komplexebb pályák esetén gyorsan átláthatatlanná válik, rontja a kód olvashatóságát, és sérti a szétválasztás elvét is.

Ennek megoldására a keretrendszer külön pályabetöltő rendszert biztosít, amelyet a `lua_core/manager.lua` modul kezel. A statikus és dinamikus pályaelemeket a `map.lua` fájlban lehet megadni.

A `meta.lua` fájlokhoz hasonlóan ezek feldolgozása is egy elkülönített környezetben, a `mapEnvironment`-ben történik. Ez a környezet valójában nem más, mint egy egyszerű Lua tábla, amely kulcs-érték párokat tartalmaz, ahol a kulcsok (például `object`, `pickup`, `fire`) névtelen függvényekre mutatnak.

Ahogy az a `meta` konfigurációknál már látható volt, a rendszer itt is alkalmazza a Lua szintaktikai sajátosságát. Mivel az objektumok létrehozásáért felelős függvények egyetlen tábla paramétert várnak, a pályatervezés során elhagyhatók a függvényhívásokat jelző zárójelek. Ennek köszönhetően az entitások az alábbi, letisztult formában definiálhatók:

Programkód 5.5. Objektumdefiníció a map.lua fájlban

```
        object {
                model = "cube.obj",
                texture = "stone.png",
                position = {0, 0, 0},
                scale = {1.0, 1.0, 1.0}
        }
```

Nem kizárólag statikus objektumok lehelyezése lehetséges ilyen módon. A `movingObject` blokk segítségével például mozgó platformok is definiálhatók. A célpozíció és az utazási idő megadásával a `mapEnvironment` automatikusan létrehozza a szükséges időzítőket és mozgást vezérlő hívásokat. A `loop` paraméternek `true` (igaz) értéket adva az objektum oda-vissza fog mozogni az eredeti pozíciója és a célkoordinátája között.

Miután a rendszer feldolgozta a `map.lua` fájlból kapott adatokat, a `mapEnvironment` függvényei a háttérben meghívják a `resourceEnvironment` megfelelő entitáslétrehozó burkolófüggvényeit, így a pályán deklarált elemek is automatikusan bekerülnek a manager entitás-nyilvántartásába.

5.5.10. Fejlesztői konzol

A fejlesztés és tesztelés során felmerült az igény, hogy valós időben is legyen lehetőség szkriptek végrehajtására. Bármilyen újrafordítás vagy programújraindítás nélkül szükség van erőforrások (*resources*) indítására, leállítására, vagy tetszőleges Lua kódok (például a kamera pozíciójának gyors áthelyezése) futtatására. Ezt a célt szolgálja a keretrendszer `lua_core/console.lua` moduljában implementált parancssoros jellegű fejlesztői konzol, amely az úgynevezett REPL (*Read-Eval-Print Loop*) koncepciót valósítja meg [7].

A konzol grafikus felületét az `onRender` eseményre feliratkozva, a `drawRectangle` és `drawText` függvények hívásával rajzolja ki a rendszer. Egy félig átlátszó rétegen jelenik meg a beviteli mező, valamint a legutóbbi tizenöt üzenetet.

A konzol bemenetkezelése kétféleképpen történik. Bár a Lua keretrendszer rendelkezik az `onKey` eseménnyel, ez a nyers, hardveres billentyűleütéseket figyeli. Ezt a konzol kizárólag a vezérlőgombok kezelésére használja: az `Enter` a parancs elküldésére, a `Backspace` a begépelt szöveg törlésére, a fel és le nyilak pedig a korábban kiadott, tárolt parancsok (*command history*) közötti navigálásra szolgálnak. A tényleges gépelés követésére a rendszer az `SDL_TEXTINPUT` eseményt alkalmazza, ami `onTextInput` néven kerül továbbításra a Lua réteg felé. Ez azért volt szükséges, mert ezzel automatikusan adhatók át a nagybetűk, valamint a speciális karakterek is.

A konzol legfőbb feladata a begépelt szöveg futtatható Lua kóddá alakítása. Mivel itt nem statikus fájlokat, hanem memóriában lévő sztringeket kell értelmezni, a modul a `load` utasítást használja [8].

Amikor a felhasználó begépel egy szöveget, a rendszer nem tudhatja előre, hogy az egy érték lekérdezése (például `getCameraPosition()`), vagy egy utasítás (például egy értékadás, `x = 10`). Ennek áthidalására a konzol elsőként kiegészíti a bemenetet egy `return` előtaggal. Ha a felhasználó lekérdezést futtatott, vagy olyan utasítást, aminek van visszatérési értéke (például egy egyszerű összeadás), a kód sikeresen lefordul.

Ha azonban a felhasználó például értékadást gépelt be, a `return` előtag hozzáfűzése (például `return x = 10`) szintaktikai hibát okoz a Lua fordítóban [8]. Ilyenkor a rendszer megpróbálja az eredeti, előtag nélküli utasítást lefordítani.

A sikeresen lefordított utasítást végül a `pcall` (*protected call*) hívással futtatja le a rendszer, a konzol pedig kiírja a kapott eredményeket, vagy éppen a hibakódokat.

A felület vizuálisan is elkülöníti a felhasználói bemeneteket (szürke szín), a sikeres kimeneteket (zöld) és a hibákat (piros), így jelentősen átláthatóbb a használata.

5.6. A keretrendszer használata

5.6.1. A menürendszer

5.7. Tesztelés

5.7.1. Funkcionális tesztelés

5.7.2. Nem-funkcionális tesztelés

6. fejezet

Összefoglalás

Források

[1] Alfred V. Aho és tsai. *Compilers: Principles, Techniques, and Tools.* 2. kiad. Pearson, 2006.

[2] Cfx.re Development Team. *FiveM Documentation.* 2024. URL: `https://docs.fivem.net/docs/` (elérés dátuma 2025).

[3] Tristan Donovan. *Replay: The History of Video Games.* Yellow Ant, 2010.

[4] David S. Ebert és tsai. *Texturing and Modeling: A Procedural Approach.* 3. kiad. Morgan Kaufmann, 2003.

[5] Christer Ericson. *Real-Time Collision Detection.* CRC Press, 2005.

[6] Kevin Forbes. *Road to the IGF: Klei Entertainment's Don't Starve.* 2014. URL: `https://www.gamedeveloper.com/design/road-to-the-igf-klei-entertainment-s-i-don-t-starve-i-` (elérés dátuma 2025).

[7] Jason Gregory. *Game Engine Architecture.* 3. kiad. CRC Press, 2018.

[8] Roberto Ierusalimschy. *Programming in Lua.* 4. kiad. Lua.org, 2016.

[9] Steven L. Kent. *The Ultimate History of Video Games.* Three Rivers Press, 2001.

[10] Eric Lengyel. *Mathematics for 3D Game Programming and Computer Graphics.* 3. kiad. Cengage Learning, 2011.

[11] Luau Development Team. *Luau: A small, fast, and embeddable programming language based on Lua.* 2024. URL: `https://github.com/luau-lang/luau` (elérés dátuma 2025).

[12] Frank D. Luna. *Introduction to 3D Game Programming with DirectX 11.* Mercury Learning és Information, 2012.

[13] Ian Millington. *AI for Games.* 3. kiad. CRC Press, 2019.

[14] Multi Theft Auto Community. *Multi Theft Auto: Scripting Introduction & Resource Management.* 2026. URL: `https://wiki.multitheftauto.com/wiki/Scripting_Introduction` (elérés dátuma 2026. 04. 04.).

[15] Multi Theft Auto Team. *Multi Theft Auto: San Andreas – Open Source Game Engine.* 2024. URL: `https://github.com/multitheftauto/mtasa-blue` (elérés dátuma 2025).

[16] Robert Nystrom. *Game Programming Patterns.* Genever Benning, 2014.

[17] Roblox Corporation. *Scripting.* 2024. URL: `https://create.roblox.com/docs/scripting` (elérés dátuma 2025).

[18] Jason Schreier. *Blood, Sweat, and Pixels: The Triumphant, Turbulent Stories Behind How Video Games Are Made.* Harper Paperbacks, 2017.

[19] Graham Sellers, Richard S. Wright és Nicholas Haemel. *OpenGL SuperBible: Comprehensive Tutorial and Reference*. 7. kiad. Addison-Wesley Professional, 2015.

[20] Simple DirectMedia Layer. *SDL2 Wiki Documentation*. 2023. URL: `https://wiki.libsdl.org/`.

[21] Simple DirectMedia Layer. *SDL_ttf 2.0 Documentation*. 2023. URL: `https://www.libsdl.org/projects/SDL_ttf/docs/`.

[22] Ian Sommerville. *Software Engineering*. 10. kiad. Pearson, 2015.

[23] Joey de Vries. *Learn OpenGL: Fundamentals of 3D Game Programming*. Kendall & Welling, 2020.

[24] James Whitehead II, Bryan McLemore és Matthew Orlando. *World of Warcraft Programming: A Guide and Reference for Creating WoW Addons*. Wiley Publishing, 2008.

CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy MarkDown fájlként kimentve).